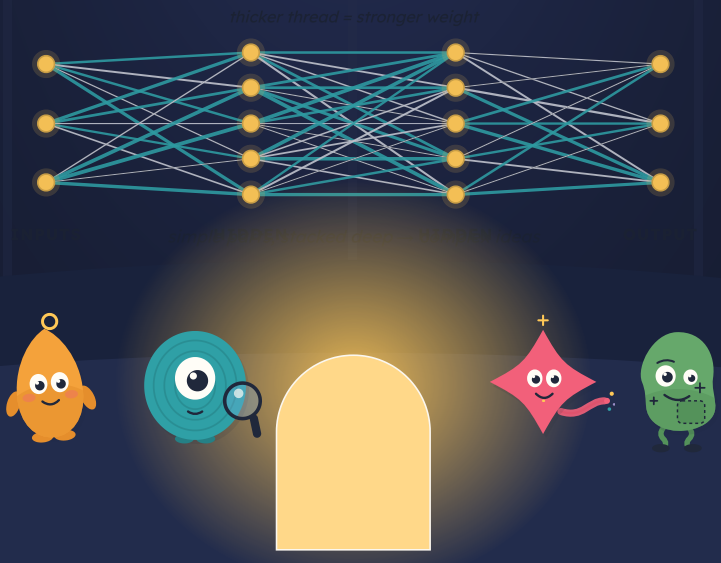
The Second Book of *Artificial* Intelligence

How Thinking Machines Actually Work

FOR INTERMEDIATE READERS · BOOK TWO OF THREE



dio.stesso

Turns out writing a book is easier than convincing someone to publish it.

So I skipped that part.

I wrote these books for my favourite people. Since you're here, I'm assuming that's you.

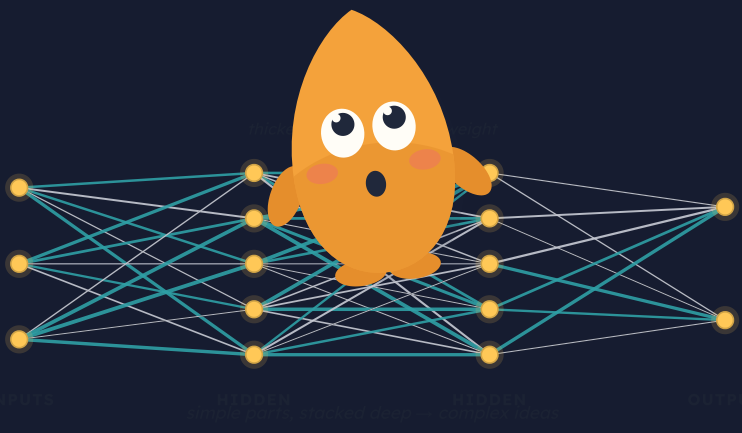
They're free until a publisher discovers my existence, offers me a contract, and starts sending royalty cheques.

Enjoy the book. If you like it, drop me a note. If you love it—or if it sparks a new idea for someone you love—Hot Wheels and LEGO remain valid currency in my kingdom.

DIO.STESSO
code.dio.stesso@gmail.com

The Second Book of *Artificial* Intelligence

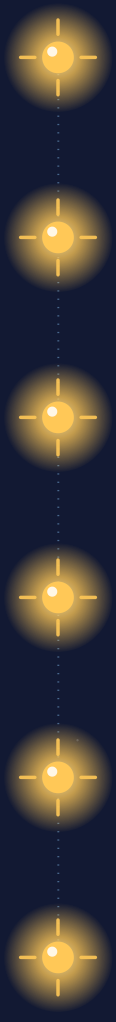
How Thinking Machines Actually Work



BY DIO.STESSO
code.dio.stesso@gmail.com

The Works

Six workshops · twenty-four mechanisms · one machine.



The Foundry · Turning the world into numbers
representation, features, vectors, embeddings, tokens, boundaries

The Training Yard · How a machine learns
loss, gradient descent, train/test, learning setups

The Lattice · Neural networks
neuron, layers, convolution, attention

The Loom & the Kiln · Language and generation
next-token, hallucination, diffusion, GANs

The Proving Grounds · How well does it really work?
accuracy, calibration, adversarial, distribution shift

The Commons · Data, people, and choices
data provenance, bias, privacy, human-in-the-loop

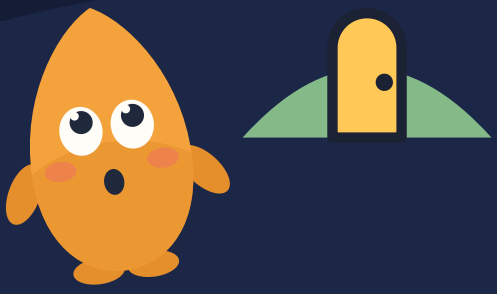
Each workshop opens a panel on how thinking machines really work.

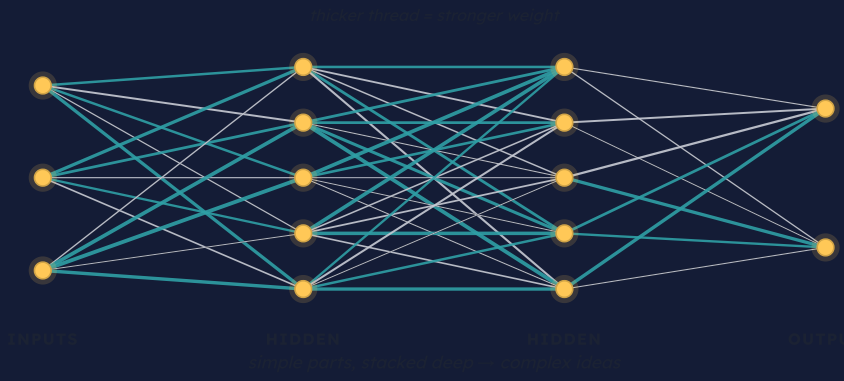
Contents

PROLOGUE

Through the Little Door

High on a hillside, pixel found a small door set into the slope — one he had walked past a hundred times without ever trying.





The Spark-Network

each Spark is one idea; here they link into layers joined by weighted threads — the structured machine you are about to open.

Pixel pushed it open. Inside was *the Works*: the inside of the machine, humming and lit. A great net of Sparks hung in the air — and it had **structure**: rows, and weighted threads, and a warm spotlight moving across it.

You may already *feel* that you understand thinking machines. This book is about seeing **how they actually run** — opening each panel in turn. Every workshop ahead builds one part of the same machine, and you do not need anything but curiosity to follow.

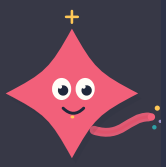
PART ONE

The Foundry

Turning the world into numbers

YOUR FOUR GUIDES

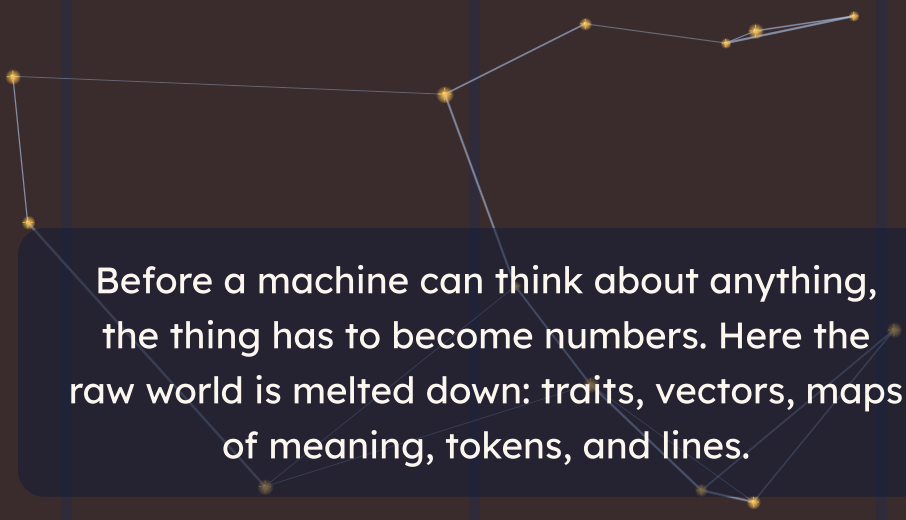
Pixel
the Apprentice — asks the sharp question



Nova
the Maker — works the Loom & Kiln

Echo
the Analyst — reads the dials

Milo
the Tester — breaks things to learn



Before a machine can think about anything, the thing has to become numbers. Here the raw world is melted down: traits, vectors, maps of meaning, tokens, and lines.

01

Numbers for Everything

A machine has never been told what a cat is. Yet it can pick the cats out of a thousand photos. How?

It cannot see fur or whiskers the way you do. A photo, to a machine, is not a picture at all. So before it can do anything clever, something quiet and strange has to happen first — the picture has to stop being a picture.

YOU ALREADY KNOW THIS

Before the machinery, three places you have already met the idea —

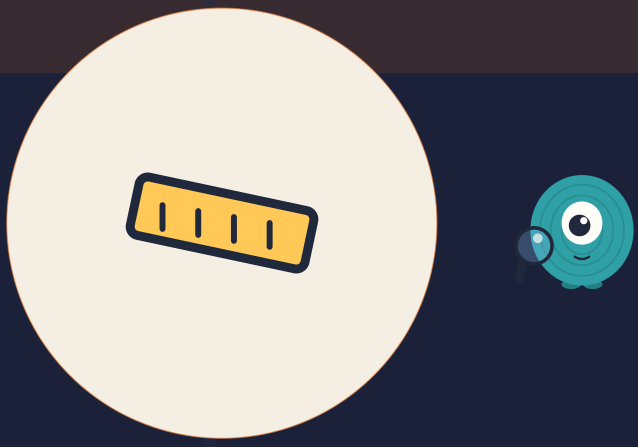
Describe a friend so someone else could spot them: height in centimetres, hair length, shoe size. You just turned a person into a short list of numbers.

A thermometer turns something you feel — warmth — into a single number you can compare, write down, and act on.

A recipe is a cake written as numbers: 200 grams of flour, 2 eggs, 18 minutes. The numbers carry enough of the cake to bake it again.

HOW IT ACTUALLY WORKS
STEP 1 OF 3

Pick what to measure



First you choose a few things worth noticing — call them **features**. For a photo it might be how bright each tiny dot is. For a fruit it might be width, weight, and bumpiness.

Read off the numbers

Each feature becomes a number. A photo is really a grid of brightness numbers, one per pixel. Nothing is added that wasn't there — you are just *reading* the world in digits.

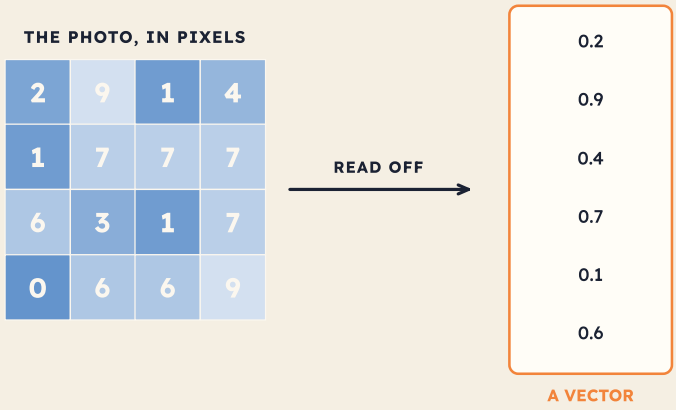
Line them up

Stack those numbers into one ordered list. That list is a **vector**: a single thing the machine can store, compare, and do maths with. A cat photo and a dog photo are now two lists to tell apart.

One thing, turned into one ordered list of numbers, has a name:

VECTOR

A vector — the machine's way of holding any single thing.



one thing → one ordered list of numbers

Your mental model. A photo becomes a grid of pixel numbers, read off into one ordered list — a vector.

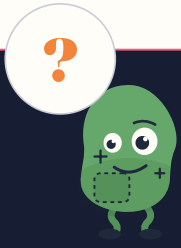
WHERE IT BREAKS

The catch is in the very first step: *you* choose which features to keep, and every feature you leave out is thrown away forever. Measure a person by height alone and you can never tell twins from strangers. Bad features quietly poison everything that comes later.

WATCH OUT

Myth: "The numbers ARE the thing."

Truth: No — the numbers are a *chosen description* of the thing. Change what you measure and you get a different vector for the very same cat. The map is not the territory.



Milo looks for where it cracks — because knowing the limits is how you use it well.

GO DEEPER · A VECTOR IS A POINT IN SPACE

Three numbers — say [width, height, weight] — can be read as a point in 3-D space. Two fruits with similar numbers sit close together; very different fruits sit far apart. Real models use hundreds or thousands of numbers, so the "space" has hundreds of directions — impossible to picture, easy to compute.

TRY THIS

Pick an object in the room. Invent five number-features for it (length, weight, number of corners…). Could a friend find the object from your five numbers alone? Which number mattered most?

You can now explain Vector.

THE INTUITION BEHIND IT

Underneath the maths is an everyday move: to handle something complicated, you pick a few things worth noticing and ignore the rest. Turning those few things into numbers is all a vector is.

WHERE THIS GOES NEXT

Later, machines learn to choose their *own* features instead of being handed them — that is where Book 3 begins.

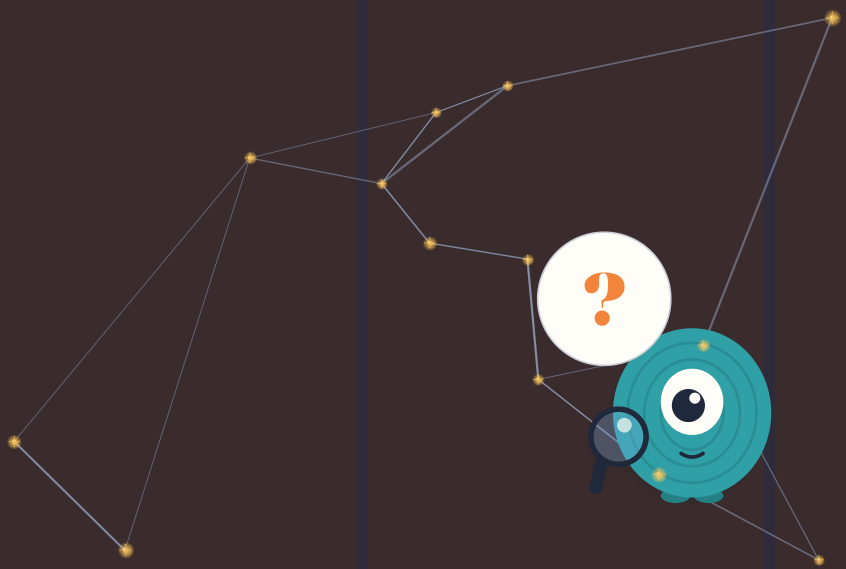
TALK IT OVER

If a machine only ever sees the numbers we choose, who is really deciding what it can and cannot notice?

02

Meaning as a Map

Type “king” and a machine somehow knows it is close to “queen”, and that both are far from “banana”. Nobody gave it a dictionary. So how does it know?



It has never tasted a banana or met a king. It only ever saw words used, again and again, in billions of sentences. Out of all that usage it built something surprising: a *place* for every word to live.

YOU ALREADY KNOW THIS

Before the machinery, three places you have already met the idea —

On a real map, cities that are near each other sit near each other on the page. Distance on the paper means something real.

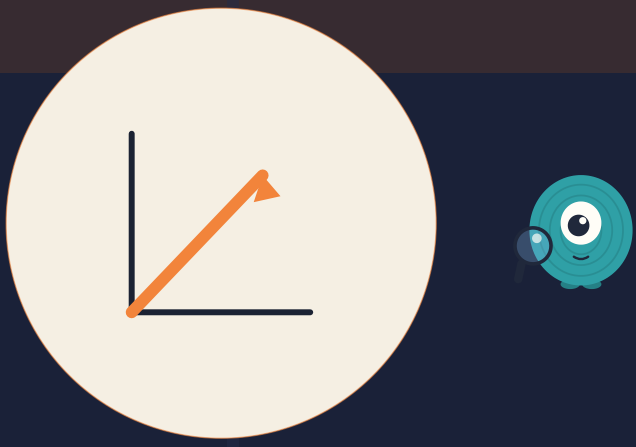
Arrange paint chips so similar colours sit together: the reds in one corner, the blues in another. Now “how different are these colours?” becomes “how far apart are they?”

Sort songs by feel and the calm ones drift to one side, the loud ones to the other. You have made a map where nearness means similarity.

Give everything a spot

Place every word as a point. The rule for placing them is simple to say: words used in similar ways should land near each other.

Let distance mean difference



Now the gaps carry meaning. “Cat” and “dog” end up neighbours; “cat” and “justice” end up far apart. The machine never needed a definition — only the arrangement.

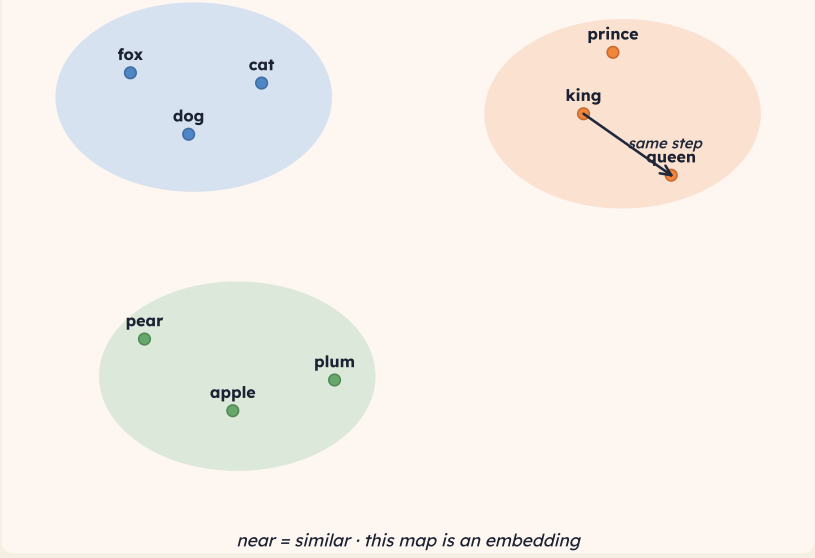
Read directions, not just dots

Even the *directions* mean something. The step from “king” to “queen” turns out to be almost the same step as “man” to “woman”. Meaning has become geometry you can measure.

A map where every word is a place and nearness means meaning has a name:

EMBEDDING

An embedding — meaning turned into a location you can measure.



Your mental model. A star-map of words: clusters of similar meanings, and directions that line up.

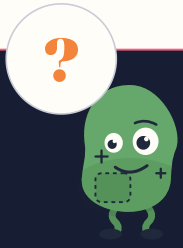
WHERE IT BREAKS

The map is built only from how people *used* words — so it soaks up the world's habits, fair and unfair alike. If texts often put “nurse” near “she” and “engineer” near “he”, the map learns that lean as if it were a fact. The arrangement is honest about usage, not about what's right.

WATCH OUT

Myth: “Close on the map means the same thing.”

Truth: Near is not equal. “Hot” and “cold” sit close — because they are used the same way — even though they mean opposites. Distance measures *similarity of use*, not sameness.



Milo looks for where it cracks — because knowing the limits is how you use it well.

GO DEEPER · SAME DIRECTION = COSINE SIMILARITY

One neat way to ask “how alike?” is to ignore how far two points are and look only at whether they point the same way from the centre. Pointing the same way (a small angle) means very similar; at right angles means unrelated. Machines call this **cosine similarity**.

TRY THIS

Draw a grid: left-to-right = “small ↔ big”, bottom-to-top = “tame ↔ wild”. Now place ant, mouse, dog, lion, whale. Whose neighbours surprised you? You just built a tiny embedding by hand.

You can now explain Embedding.

THE INTUITION BEHIND IT

You already judge similarity by feel — which songs, colours, or animals "go together". An embedding turns that feeling into a real, measurable distance, built on the vectors from Chapter 1.

WHERE THIS GOES NEXT

Real maps have hundreds of directions at once — a *latent space* you can travel through. Book 3 lives there.

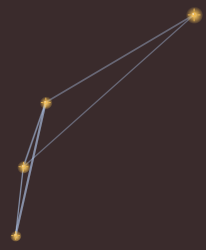
TALK IT OVER

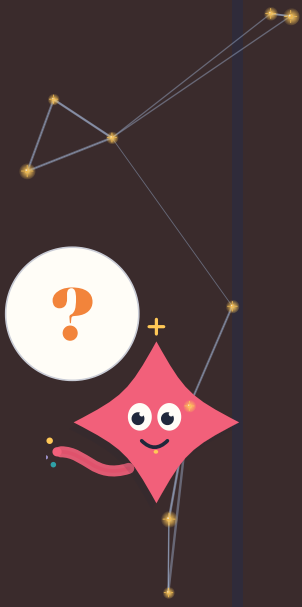
If the word-map quietly copies the world's unfair habits, is the machine being biased — or just honest about the texts we fed it?

03

Words into Tokens

Does a machine read letters, like a beginner? Whole words, like you? It turns out it reads something in between — and that choice explains a lot of its quirks.





Before a single clever thing happens to a sentence, the sentence is chopped up. Not into letters, not always into words, but into pieces of a very particular size. Get curious about those pieces and a lot of strange machine behaviour suddenly makes sense.

YOU ALREADY KNOW THIS

Before the machinery, three places you have already met the idea —

Think of building a word out of LEGO. “Unhappiness” isn't one smooth brick — it snaps into “un”, “happi”, “ness”. Familiar chunks, reused.

Cut a long sentence into the smallest pieces that still appear often. Common words stay whole; rare words shatter into bits the machine has seen before.

A new or made-up word — like a wild username — gets sliced into letters, because the machine has no familiar chunk that fits it.

HOW IT ACTUALLY WORKS
STEP 1 OF 3

Build a chunk dictionary

From mountains of text, the machine finds the chunks that show up most. Whole common words, common word-parts, and single letters for everything else. These chunks are **tokens**.

Slice every input

Any sentence you give it is first cut into those tokens — sub-word pieces, not words. “Cats” might be “cat” + “s”; “unhappiness” becomes three tokens.

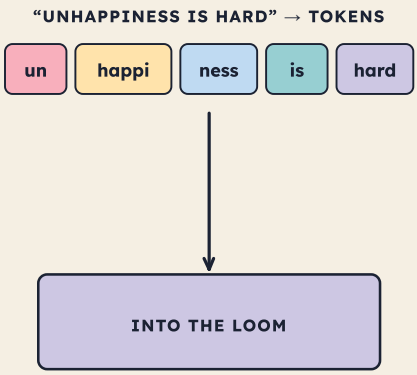
Feed the pieces in

Only then does the machine go to work, one token at a time. Everything it does — predicting, focusing, answering — happens on tokens, never on letters or whole words.

The sub-word pieces a machine actually reads have a name:

TOKEN

Tokens — the real units of machine language.



rare words split into smaller pieces

Your mental model. A sentence sliced into labelled token-tiles, fed into the Loom; rare words split into smaller pieces.

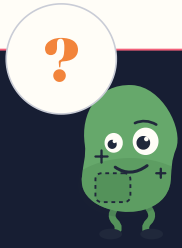
WHERE IT BREAKS

Because the pieces are chosen by frequency, odd things happen at the edges. A rare name fragments into nonsense chunks; counting letters is hard when the machine never sees letters; a space in the wrong place can change the tokens entirely. Many “silly” mistakes are really tokenisation showing through.

WATCH OUT

Myth: “One word equals one token.”

Truth: Often, but not always. Short common words are one token; long or rare words are several; and a token can even be part of a word or just a space. The machine counts in tokens, not words.



Milo looks for where it cracks — because knowing the limits is how you use it well.

GO DEEPER · WHY SUB-WORDS WIN

Whole-word pieces can't handle words the machine has never seen. Single letters can — but then sentences become enormously long and the machine forgets the start by the end. Sub-word tokens are the compromise: small enough to cover any new word, big enough to stay short.

TRY THIS

Split “antidisestablishmentarianism” into chunks you'd recognise (“anti”, “dis”, “establish”…). Then try a friend's surname. Which words break into the most pieces, and why?

You can now explain Token.

THE INTUITION BEHIND IT

Before a machine can worry about which words go together, it has to decide what a “piece” of language even is. Tokens are those pieces — the raw thread the Word-Loom of Chapter 13 later weaves.

WHERE THIS GOES NEXT

At today's scale, tokenising trillions of words — and across every language — is its own deep problem. More in Book 3.

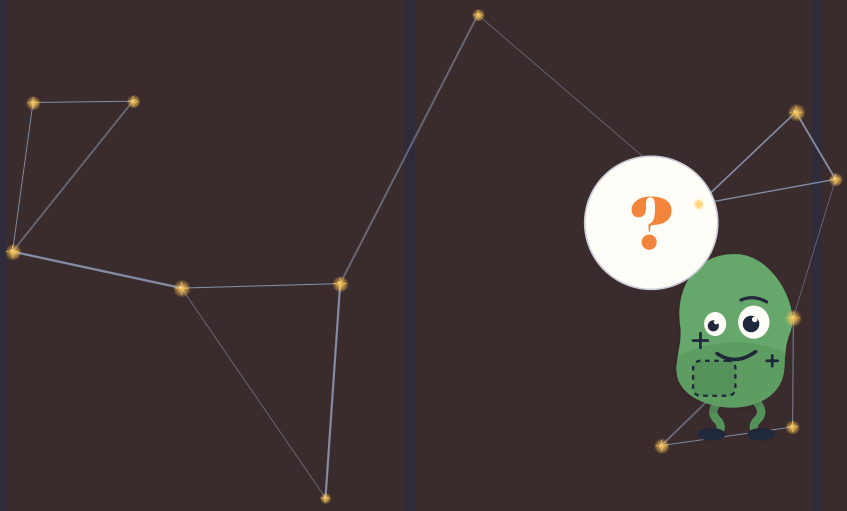
TALK IT OVER

If a machine literally never sees individual letters, should we be surprised when it miscounts the letters in a word?

04

Drawing the Line

Cat or dog? You decide in a heartbeat. A machine has no idea what either is — yet it can sort them too. What is it actually doing?

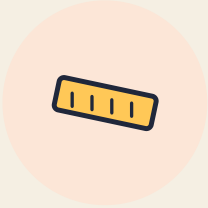


Once things are numbers (Chapter 1) living on a map (Chapter 2), sorting them stops being about fur and barks. It becomes something almost geometric: a question of *which side of a line* a point lands on.

YOU ALREADY KNOW THIS

Before the machinery, three places you have already met the idea —

Tip a bag of fruit onto a table and arrange it by two traits — size left-to-right, redness bottom-to-top. The apples drift to one region, the limes to another.



Now lay a string across the table so apples are on one side, limes on the other. New fruit? Whichever side of the string it lands, that's your guess.

Sorting happy from sad faces works the same way once each face is a few numbers: find the line that best keeps the two groups apart.

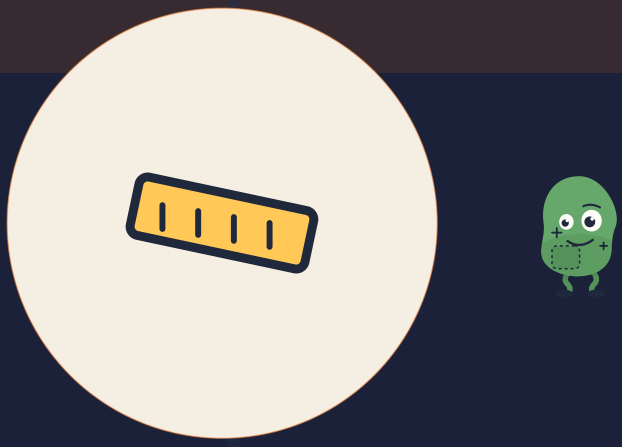
HOW IT ACTUALLY WORKS
STEP 1 OF 3

Spread the points out

Each example becomes a point in feature-space — the map from Chapter 2. Cats cluster here, dogs cluster there, with some messy middle.

HOW IT ACTUALLY WORKS
STEP 2 OF 3

Find the dividing line



The machine searches for the line — or curve — that best separates the clusters. This divider is the **decision boundary**.

HOW IT ACTUALLY WORKS
STEP 3 OF 3

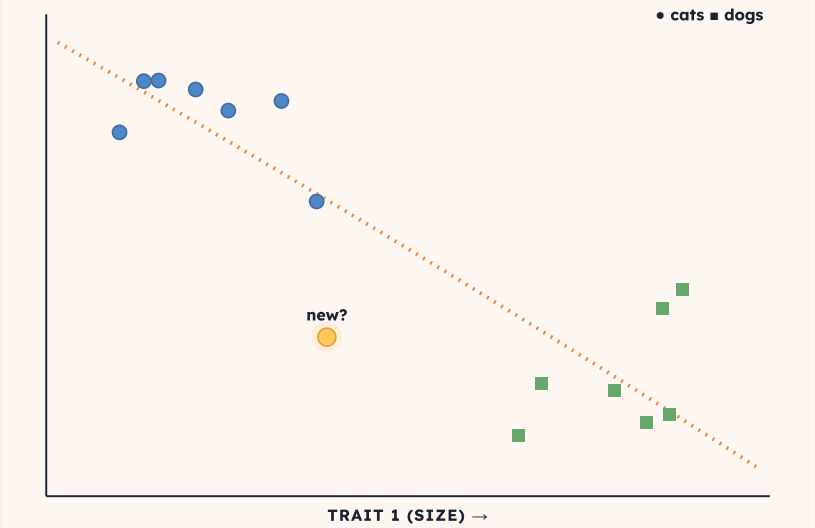
Classify by side

To label a brand-new point, you don't compare it to every example. You just ask: which side of the boundary did it fall? That side is your answer.

The line that splits one group from another has a name:

DECISION BOUNDARY

A decision boundary — sorting is just which side you're on.



sorting = which side of the boundary you land on

Your mental model. A scatter of points with a curved boundary; a new point lands on one side and gets that label.

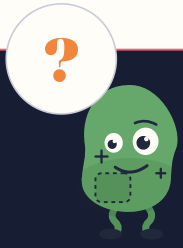
WHERE IT BREAKS

Real groups overlap. Some cats look like dogs, and no single line can split a tangle cleanly — push it to separate everyone and it starts memorising freckles instead of learning the rule. A straight line is often too simple, which is exactly why we'll later stack many of them.

WATCH OUT

Myth: "The line is always straight, and always right."

Truth: Boundaries can curve, and near the line the machine is barely guessing. A point right on the boundary is a coin-flip — confidence should drop there, not stay high.



Milo looks for where it cracks — because knowing the limits is how you use it well.

GO DEEPER · MARGINS AND CONFIDENCE

The best boundary doesn't just separate the groups — it leaves the widest gap, or **margin**, on either side. Points far from the line are easy calls; points hugging it are uncertain. A model's confidence is really "how far from the boundary am I?"

TRY THIS

On graph paper, plot 5 "cats" and 5 "dogs" as dots in two clusters. Draw the single line that best splits them. Now add one dot in the messy middle — could one straight line ever be fair to it?

You can now explain Decision Boundary.

THE INTUITION BEHIND IT

Whenever you sort things into groups, you are really drawing a line between them. Place your examples in the number-space from Chapters 1–2 and that line becomes a decision boundary — the engine inside every “cat or dog?”.

WHERE THIS GOES NEXT

A single neuron, coming next, computes exactly one of these lines. Stack them and the lines start to bend.

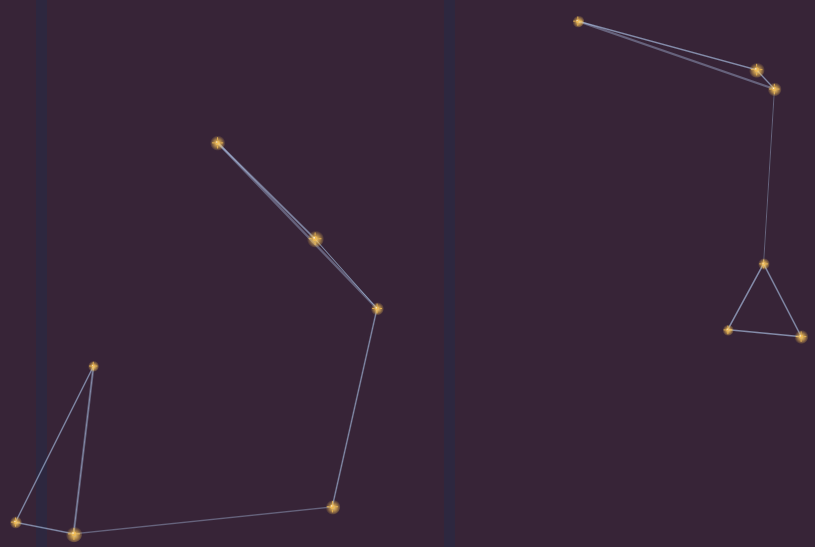
TALK IT OVER

When a machine is forced to pick a side for a point sitting right on the line, is its confident answer honest?

PART TWO

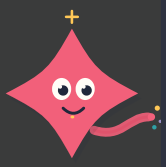
The Training Yard

How a machine learns



YOUR FOUR GUIDES

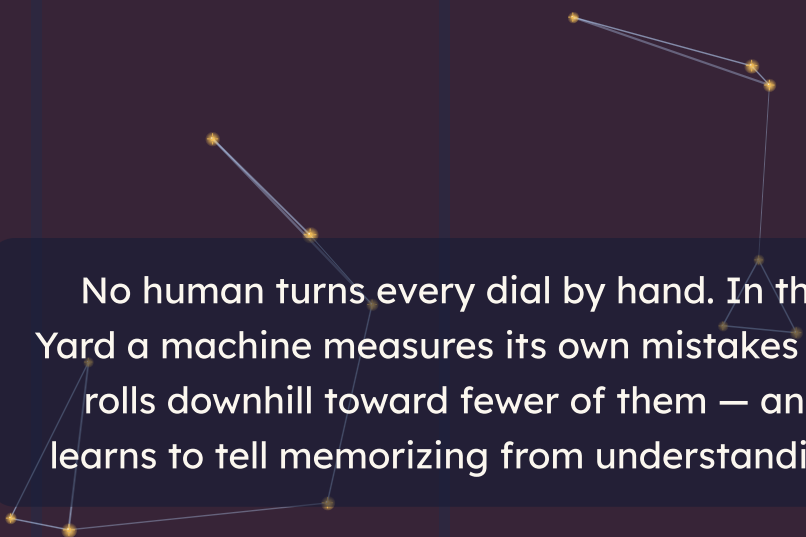
Pixel
the Apprentice — asks the sharp question



Nova
the Maker — works the Loom & Kiln

Echo
the Analyst — reads the dials

Milo
the Tester — breaks things to learn

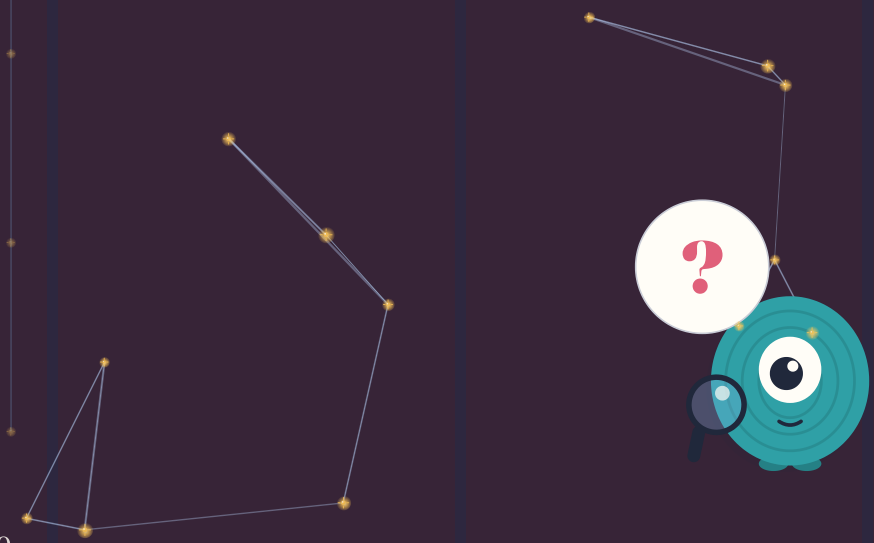


No human turns every dial by hand. In the Yard a machine measures its own mistakes and rolls downhill toward fewer of them — and learns to tell memorizing from understanding.

05

The Mistake Meter

A machine has nobody watching over its shoulder. So how does it even know when it's wrong — let alone get better?



In the Foundry we turned the world into numbers. Learning needs one more number, and it's the most important one in this whole book: a single score for *how wrong* the machine just was.

YOU ALREADY KNOW THIS

Before the machinery, three places you have already met the idea —

Throw a dart. You don't just say "miss" — you see *how far* from the bullseye. Two centimetres off is a better throw than twenty.

Guess a friend's age. Off by one year is a small mistake; off by thirty is a big one. The size of the gap is the lesson.

Predict tomorrow's temperature. The difference between your guess and the real reading tells you exactly how much to adjust.

HOW IT ACTUALLY WORKS
STEP 1 OF 3

Compare guess to truth

For each example, line up the machine's guess against the real answer. The gap between them is the raw mistake.

Turn the gap into a number

Measure every gap and combine them into one score. Small gaps → small score; big gaps → big score. That single number is the **loss**.

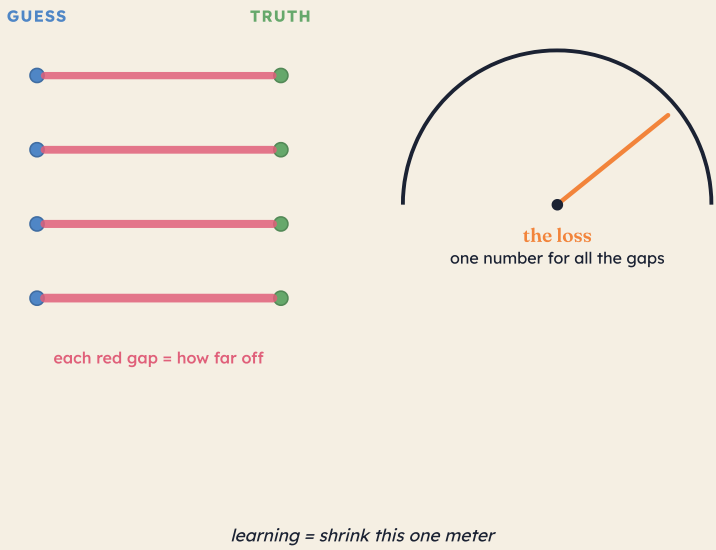
Make it the target

Now learning has a clear job: change the dials until the loss goes *down*. The whole Training Yard exists to shrink this one meter.

The single number for how wrong a machine is has a name:

LOSS

Loss — the meter every learning machine tries to shrink.



Your mental model. Guesses beside truths; each red gap is an error, summed into one number — the loss.

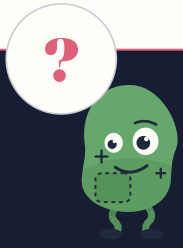
WHERE IT BREAKS

You get exactly what you measure — and nothing else. Choose a loss that ignores rare cases and the machine will cheerfully ignore them too. A low loss means “good at the thing we scored”, which is not always “good at the thing we wanted”.

WATCH OUT

Myth: “Low loss means the answer is good and true.”

Truth: Low loss only means low on *this* measure, over *this* data. Score the wrong thing well and you've taught the wrong lesson perfectly.



Milo looks for where it cracks — because knowing the limits is how you use it well.

GO DEEPER · WHY WE AVERAGE OVER MANY

Loss is measured over lots of examples and averaged, not judged on one lucky guess. A model that nails one example and fails ten has high average loss — which is what we want, because we care about the whole job, not one moment.

TRY THIS

Play “guess the number” 1–100 with a friend. After each guess, write the gap (guess minus answer). Add the gaps up. Try to make that total *shrink* over five rounds — you just minimised a loss.

You can now explain Loss.

THE INTUITION BEHIND IT

You already know a near-miss beats a wild miss — the size of the gap is the lesson. Loss is that everyday sense of "how far off was I?" sharpened into one number a machine can act on.

WHERE THIS GOES NEXT

Choosing *what* to score — the reward — is one of the hardest problems in all of AI. Book 3 calls it the alignment problem.

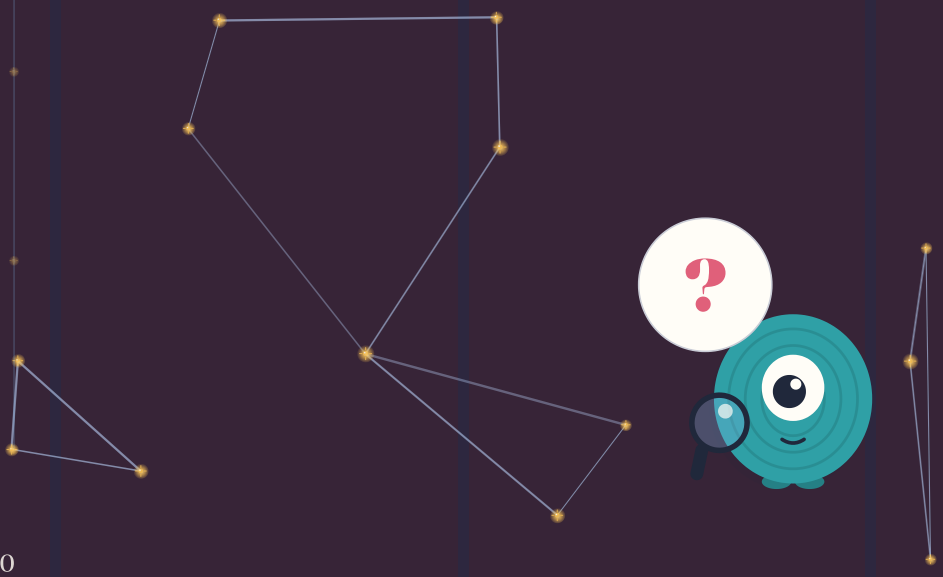
TALK IT OVER

If a machine becomes brilliant at the exact thing we measured, but it was the wrong thing to measure, whose mistake is that?

06

Rolling Downhill

A big model has millions of dials. Nobody turns them by hand. So how do millions of numbers tune themselves toward the right answer?



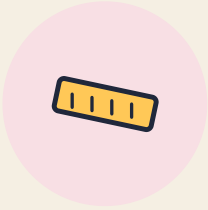
We have a meter to shrink — the loss from Chapter 5. The trick for shrinking it is so simple a ball can do it, and so powerful it trains nearly every AI you've ever heard of.

YOU ALREADY KNOW THIS

Before the machinery, three places you have already met the idea —

Stuck in a valley in thick fog, eyes shut. You can't see the bottom — but you can *feel* which way the ground slopes under your feet.

You take a small step downhill. Feel again. Step again. You don't need a map of the whole valley — only the slope right where you stand.

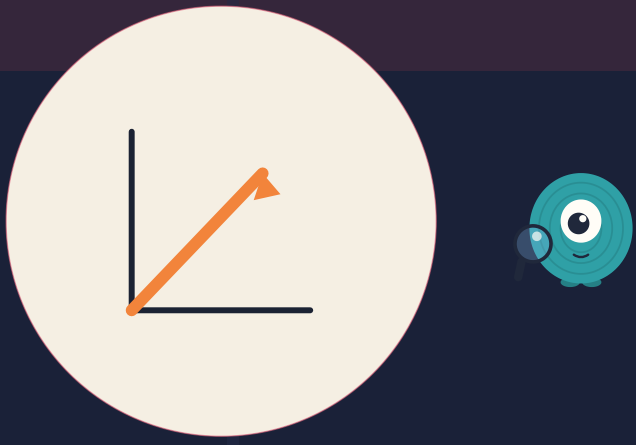


Tune a guitar by ear: pluck, hear if it's sharp or flat, nudge the peg the right way, repeat. Each tiny correction is guided by which way is “better”.

Measure the slope

At the current setting of the dials, the machine works out which way the loss goes *down* fastest. That direction-of-steepest-decrease is the **gradient**.

Take a step that way



Nudge every dial a little in the downhill direction. The loss drops a bit. This nudge-the-dials step is **gradient descent**.

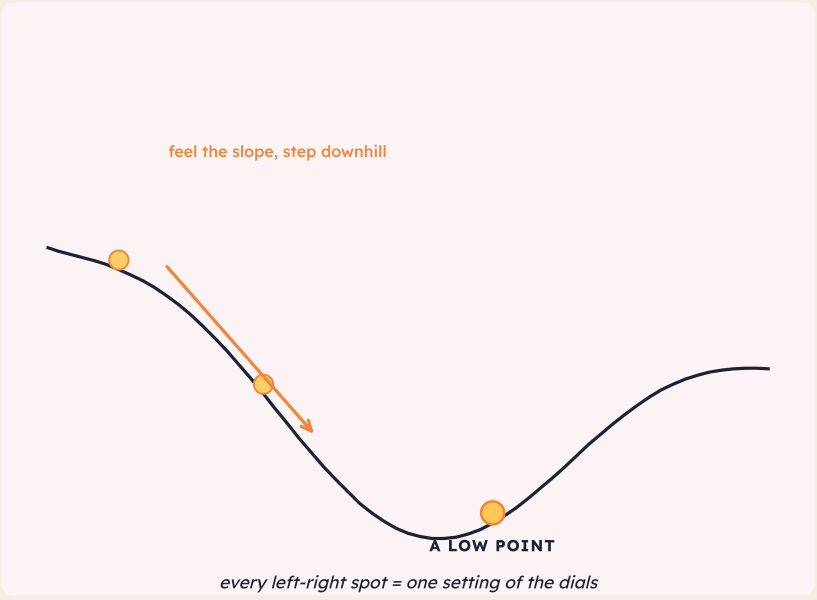
Repeat, thousands of times

Slope, step, slope, step — over and over. No human turns a single dial. The machine simply keeps walking downhill until the loss stops falling.

Shrinking the loss by feeling the slope and stepping downhill has a name:

GRADIENT DESCENT

Gradient descent — how machines tune millions of dials by themselves.



Your mental model. A loss landscape with a ball feeling the slope and stepping toward a low point.

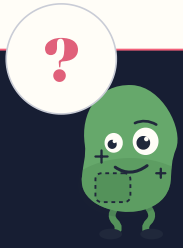
WHERE IT BREAKS

Downhill is not the same as “lowest”. The ball can settle in a small dip and never find the deepest valley. Steps too big overshoot and bounce; too small and it crawls forever. And the real landscape isn't a 2-D valley — it has *millions* of directions at once.

WATCH OUT

Myth: “Gradient descent finds the perfect answer.”

Truth: It finds a *low* point near where it started, not necessarily the lowest one anywhere. Good enough, usually — perfect, almost never.



Milo looks for where it cracks — because knowing the limits is how you use it well.

GO DEEPER · THE LEARNING RATE IS THE STEP SIZE

How big a step to take each time is called the **learning rate**. Too large and the ball leaps over the valley and never settles; too small and training takes forever. Choosing it well is part science, part craft.

TRY THIS

Hide a coin and have a friend find it by warmer/colder hints only — never by pointing. Notice they reach it by always stepping toward “warmer”, exactly like following a slope downhill.

You can now explain Gradient Descent.

THE INTUITION BEHIND IT

Correcting a mistake by nudging toward “better” is what you do every time you tune an instrument or steer toward “warmer”. Gradient descent is that same nudge — shrinking the loss from Chapter 5 — across millions of dials at once.

WHERE THIS GOES NEXT

Training the largest models means doing this across thousands of computers for months. Book 3 visits that scale.

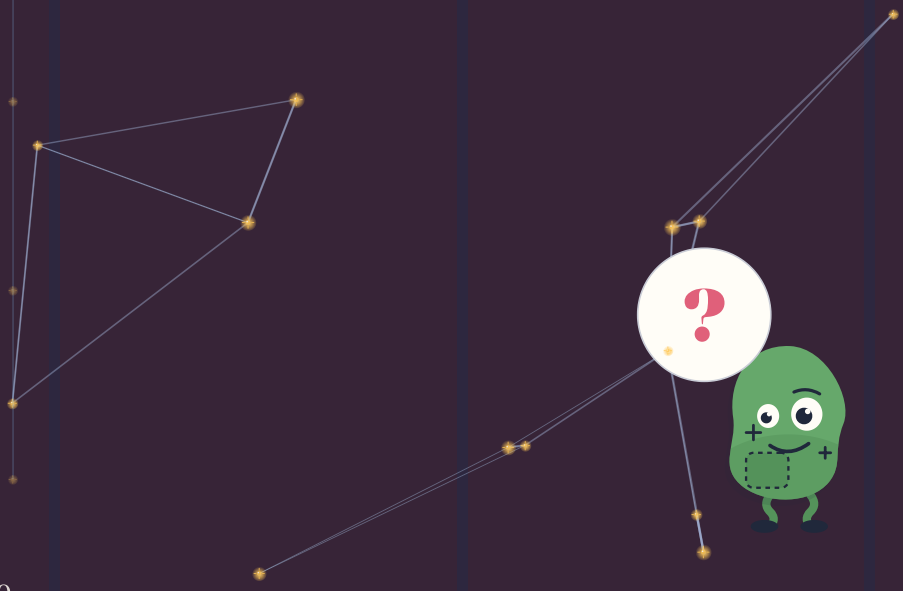
TALK IT OVER

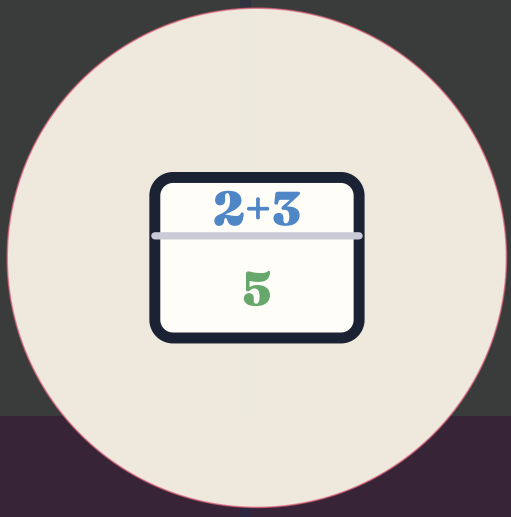
If a model gets stuck in a “good enough” dip instead of the best answer, how would we ever know what it missed?

07

Memorize or Understand?

A student aces every practice sheet, then fails the real exam. Machines do this too — and learning to spot it is half of making them work.





Shrinking the loss (Chapters 5–6) sounds like pure good news. But there's a trap hiding inside it, and it's the same trap that catches the cramming student the night before a test.

YOU ALREADY KNOW THIS

Before the machinery, three places you have already met the idea —

One kid learns the *ideas* behind the maths; another memorises the exact answers on the practice sheet. Both score full marks — until the questions change.

A child who has only ever met poodles decides “dog” means “curly and small”. Meet a Great Dane and the rule falls apart.

Practising the leaked exam paper gets you a perfect mock score that means nothing, because you learned *those* questions, not the subject.

HOW IT ACTUALLY WORKS
STEP 1 OF 3

Split your data

Keep some examples hidden. Learn on the **training** set, tune choices on a **validation** set, and judge fairness only on a **test** set the machine has never seen.

HOW IT ACTUALLY WORKS
STEP 2 OF 3

Watch two scores, not one

Track the loss on training data *and* on held-out data. For a while both fall together — the machine is genuinely learning.

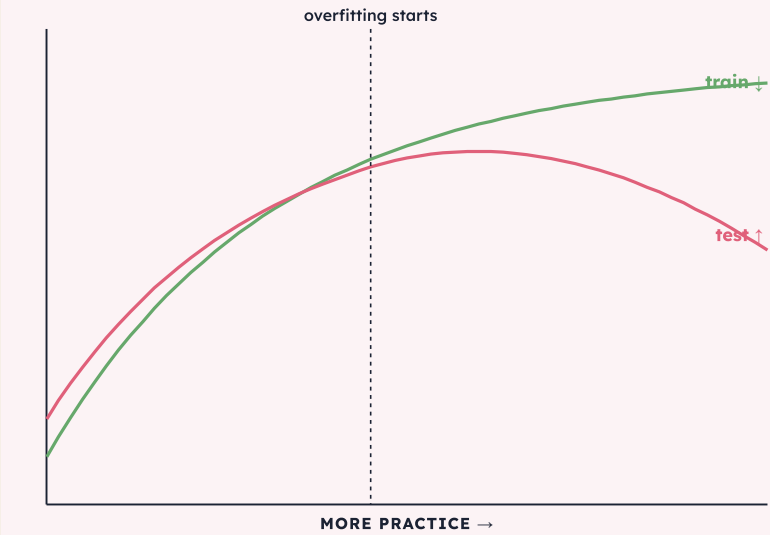
Catch the moment they split

Then training loss keeps dropping while test loss starts *rising*. That gap is **overfitting**: the machine has started memorising the practice set instead of learning the idea.

Memorising the practice instead of learning the idea has a name:

OVERFITTING

Overfitting — and beating it is called generalising.



memorising the practice instead of learning the idea

Your mental model. Two curves: training loss keeps falling while test loss turns and rises — overfitting begins.

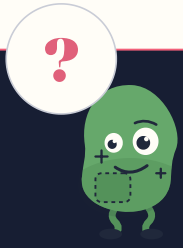
WHERE IT BREAKS

Even a model that generalises beautifully still only knows the world it trained on. Show it something genuinely new — a season it never saw, a phrase that didn't exist yet — and it can fail without warning. Passing the test set is not a promise about the future.

WATCH OUT

Myth: "More training is always better."

Truth: Past a certain point, more training makes a model *worse* at real cases as it memorises noise. Knowing when to stop is part of the skill.



Milo looks for where it cracks — because knowing the limits is how you use it well.

GO DEEPER · REGULARISATION = KEEP IT SIMPLE

One cure for overfitting is to gently push the model toward **simpler** explanations — fewer wild wiggles in the boundary. This nudge is called **regularisation**: a thumb on the scale that says “don't get fancy unless the data really demands it”.

TRY THIS

Memorise the answers to 5 specific sums (7×8=56…). Then have a friend ask 5 *different* sums. Memorising won't help — only understanding times-tables will. That's the difference in one minute.

You can now explain Overfitting.

THE INTUITION BEHIND IT

Learning from examples only works if you grasp the idea, not just the examples — the trap that catches the student who memorised the practice paper. We held data back to catch it here, and it returns, larger, as distribution shift in Chapter 20.

WHERE THIS GOES NEXT

When a benchmark's answers leak into training, models "ace" it without learning anything — a problem we meet again in the Proving Grounds, and again in Book 3.

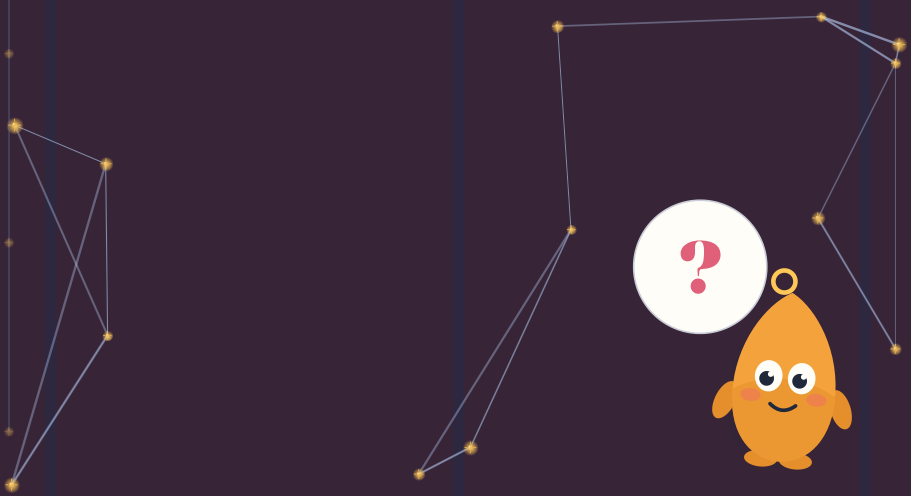
TALK IT OVER

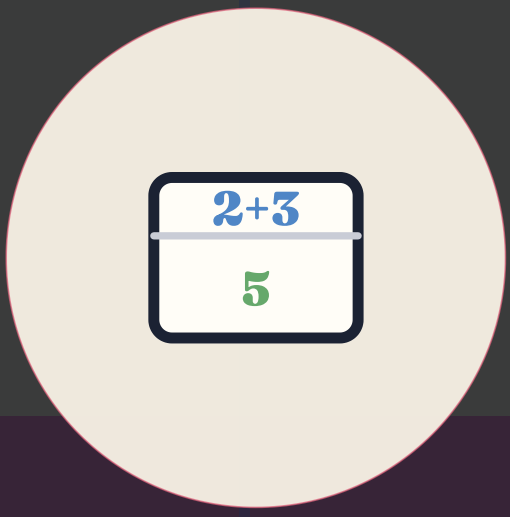
If a model scores perfectly on every test we can think of, can we ever be sure it understood — or only that we ran out of new questions?

08

Four Ways to Learn

Can you learn something with no teacher at all — nobody to mark your answers? It sounds impossible. It is exactly how today's biggest models learned.



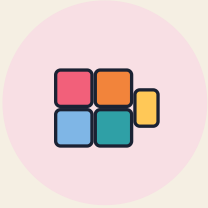


So far the machine learned from examples with answers attached. But answers are expensive, and sometimes there are none. It turns out there isn't one way to learn — there are four, each suited to a different situation.

YOU ALREADY KNOW THIS

Before the machinery, three places you have already met the idea —

Supervised: flashcards with the answer on the back. You see a question, guess, and check. Great — when someone has labelled every card.



Unsupervised: tidy a messy room with no instructions, just by grouping things that feel alike. No answers given — you find the structure yourself.

Self-supervised: cover a word in a sentence and guess it from the rest. The text quietly *becomes its own answer key*, free of charge.

HOW IT ACTUALLY WORKS
STEP 1 OF 3

Match the setup to the data

If you have labelled answers, learn supervised. If you only have raw stuff, learn unsupervised. If the data can hide part of itself, learn self-supervised.

Learn from rewards

Reinforcement: like training a dog with treats, the machine tries things and gets a reward for good outcomes — no answer key, just better and worse results over time.

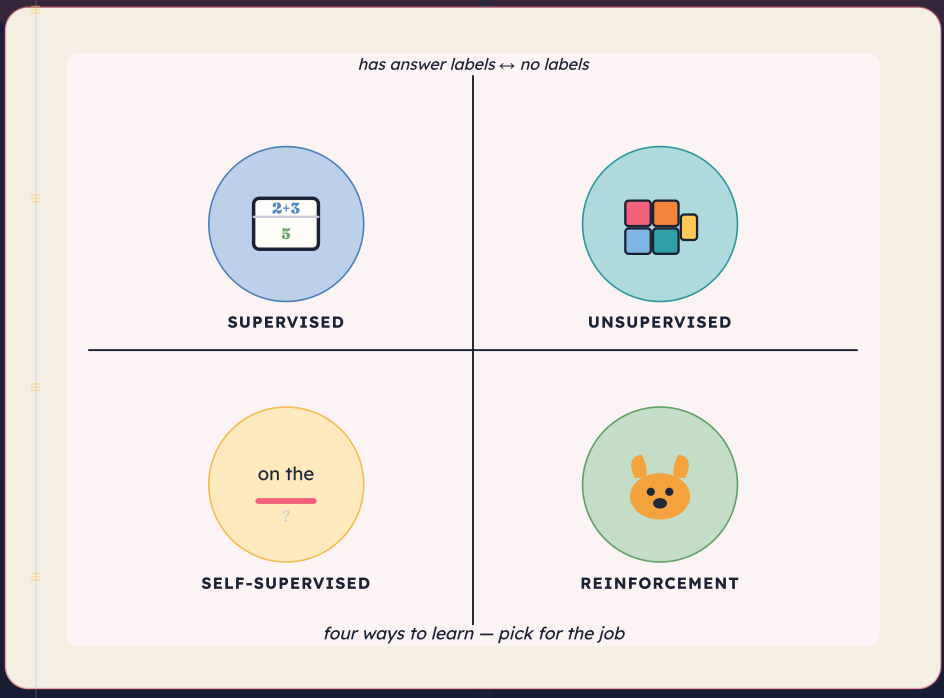
Pick for the job

Each setup needs different things and is good at different jobs. The art is choosing the right one — and often, combining them.

Learning by hiding part of the data and predicting it has a name:

SELF-SUPERVISED

Self-supervised — the fourth way, and the one that taught today's models.



Your mental model. Four quadrants — supervised, unsupervised, self-supervised, reinforcement — each with its own learner.

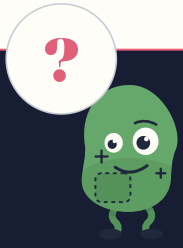
WHERE IT BREAKS

Each way has a price. Supervised learning needs humans to label mountains of data — slow and costly. Reinforcement needs a reward that's fiendishly hard to design without it being gamed. There is no free lunch; you trade one difficulty for another.

WATCH OUT

Myth: "Unsupervised learning has no goal."

Truth: It has a goal — find the hidden structure — just not labelled answers. "No teacher" isn't "no purpose".



Milo looks for where it cracks — because knowing the limits is how you use it well.

GO DEEPER · WHY SELF-SUPERVISION CHANGED EVERYTHING

The web is full of text and images but almost no labels. Self-supervision turns that flood into free training: hide a word, predict it; hide a patch, fill it in. Suddenly *all* the world's raw data becomes a teacher. That unlock is why modern models could grow so large.

TRY THIS

Take any sentence from this page, cover one word with your thumb, and predict it from the rest. You were right surprisingly often, weren't you? That's self-supervised learning, running in your head.

You can now explain Self-Supervised.

THE INTUITION BEHIND IT

Practising to get good is only one way to learn. Once you notice that data can hide part of itself and mark its own answers, you have the trick — self-supervision — that fed the giant models of Part IV.

WHERE THIS GOES NEXT

Modern chatbots add a final human-feedback stage — people rank answers — called RLHF. That story is Book 3's.

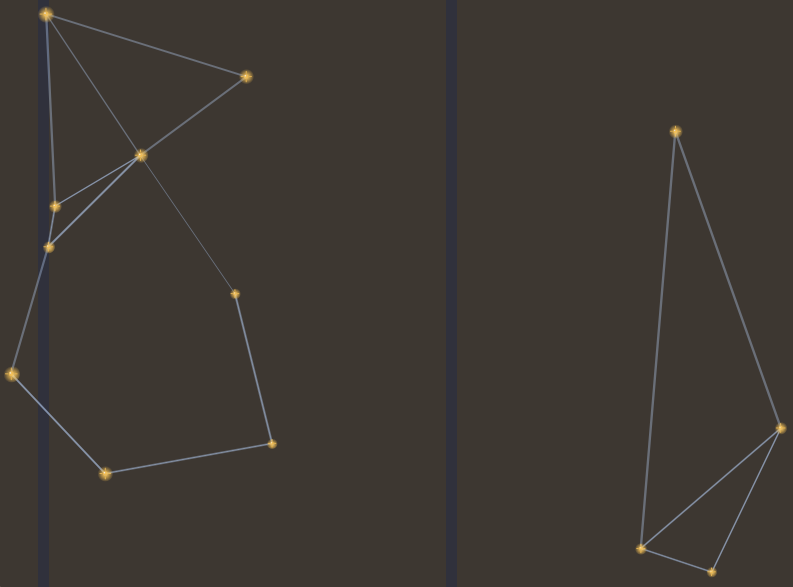
TALK IT OVER

If a machine teaches itself from the whole internet with no one checking the lessons, what might it quietly learn that no teacher would have taught?

PART THREE

The Lattice

Neural networks



YOUR FOUR GUIDES

Pixel
the Apprentice — asks the sharp question

Nova
the Maker — works the Loom & Kiln

Echo
the Analyst — reads the dials

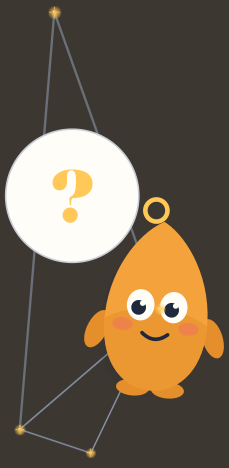
Milo
the Tester — breaks things to learn

A vast hall of glowing Spark-rows joined by weighted threads. From one tiny decider, to deep stacks, to machines that see and machines that focus.

09

The Tiny Decider

What is the smallest possible thing that can make a decision? Strip a thinking machine down to one piece — what's left?



In the Foundry we drew a single line to sort things (Chapter 4). In the Lattice we discover that this line is computed by one tiny part — and that everything grand a neural network does is built from millions of copies of it.

YOU ALREADY KNOW THIS

Before the machinery, three places you have already met the idea —

Deciding whether to take an umbrella: dark clouds matter a lot, a slight breeze a little, yesterday's weather hardly at all. You weigh the clues.

Each clue gets an importance — a **weight**. You add up the weighted clues, and if the total crosses a line, you act.

A committee vote where some members count for more than others: tally the weighted votes, and above a threshold the motion passes.

HOW IT ACTUALLY WORKS
STEP 1 OF 3

Weight each input

A **neuron** takes several inputs, each multiplied by its own weight — big weight for an important clue, tiny for a useless one.

Add them up

It sums all the weighted inputs into one total. That total says how strongly the evidence points one way.

HOW IT ACTUALLY WORKS
STEP 3 OF 3

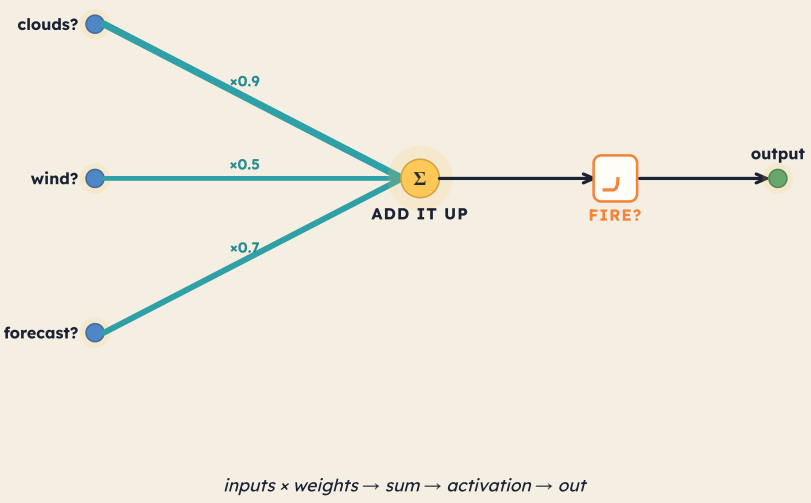
Fire or stay quiet

Finally it passes the total through an **activation**: a little gate that decides whether — and how strongly — the neuron “fires”. That output becomes an input to the next neuron.

The smallest part that can decide has a name:

NEURON

A neuron — weighted inputs, summed, then fired through an activation.



inputs × weights → sum → activation → out

Your mental model. One neuron: inputs × weights → sum → activation → output.

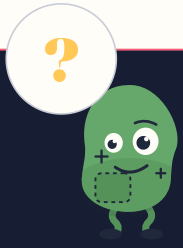
WHERE IT BREAKS

On its own, a single neuron is weak. The line it draws is straight — it can split "clearly cloudy" from "clearly clear", but it can't handle anything tangled. One neuron is barely a decider; its power only appears when you wire many together.

WATCH OUT

Myth: "A neuron is exactly like a brain cell."

Truth: It's a loose *metaphor*. A real brain cell is wildly more complex and behaves differently. The machine's "neuron" is a small piece of arithmetic that borrowed the name.



Milo looks for where it cracks — because knowing the limits is how you use it well.

GO DEEPER · WHY THE ACTIVATION MUST BEND

If a neuron just added things up, stacking thousands of them would still only ever draw straight lines. The **activation** bends the output — even a simple bend, like “negatives become zero” — and that bend is what lets stacked neurons curve their boundaries around tangled data.

TRY THIS

Decide “should I go outside?” by giving 3 clues a weight out of 10 (sunny +8, homework −5, friend's waiting +6). Add them. Set a threshold of, say, 5. Did you “fire”? You just ran a neuron.

You can now explain Neuron.

THE INTUITION BEHIND IT

The straight line from Chapter 4 is exactly what one neuron computes. Now we know what was drawing it.

WHERE THIS GOES NEXT

Replace fixed weights with weights that *change per word* and you get attention — three chapters from now, and the heart of Book 3.

TALK IT OVER

If one neuron is just weighted adding plus a bend, where does the "intelligence" of a whole network actually come from?

10

Stacking Deciders

One neuron draws one simple line. So how do machines built from neurons handle something as hard as recognising a face, or a sentence?

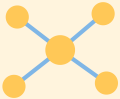
The answer is the big idea of the whole Lattice, and it's the same trick a sandcastle uses: you don't solve a hard thing all at once. You build it in *layers*, each resting on the one below.

YOU ALREADY KNOW THIS

Before the machinery, three places you have already met the idea —

An assembly line: one station adds wheels, the next adds doors, the next paints. No station builds the whole car — each adds one stage to the last.

In vision: early layers find edges, the next combine edges into shapes, the next combine shapes into eyes and noses, the last says “a face”.



A committee of committees: small groups settle small questions, then pass their answers up to groups that settle bigger ones.

Arrange neurons in rows

Line the neurons up in **layers**: an input layer, one or more hidden layers, an output layer. Signals flow along weighted threads from one row to the next.

Let each layer build on the last

Each layer takes the patterns the previous layer found and combines them into something richer. Simple features early; complex ideas late.

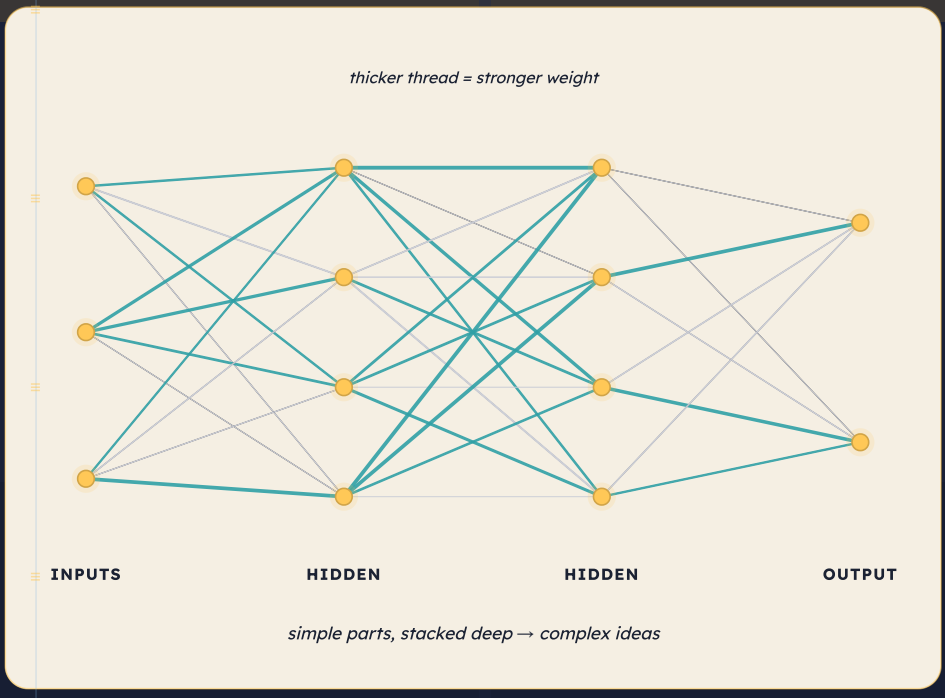
Go deep

Stack many layers and the network can represent genuinely complex things. Many layers is what “deep” means — hence **deep learning**.

Stacking many layers of neurons to learn complex things has a name:

DEEP LEARNING

Deep learning — depth is what lets simple parts handle hard problems.



Your mental model. The Lattice: input → hidden layers → output, signals flowing along weighted threads.

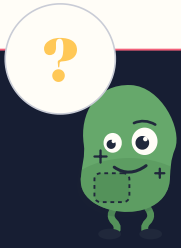
WHERE IT BREAKS

Depth is not free. More layers need more data and far more computing power, and the deeper a network gets the harder it is to see *why* it decided anything — it becomes a box we can test but not easily read. Power and mystery grow together.

WATCH OUT

Myth: "More layers always means smarter."

Truth: Past a point, extra depth just adds cost, instability, and overfitting without adding skill. The right depth fits the problem; bigger isn't automatically better.



Milo looks for where it cracks — because knowing the limits is how you use it well.

GO DEEPER · WHAT A HIDDEN LAYER "REPRESENTS"

Nobody tells the hidden layers what to look for. Yet when we peek, early layers often end up detecting edges, later ones textures, later still whole objects — features the network *invented* for itself because they helped lower the loss. That self-built ladder of features is the quiet magic of depth.

TRY THIS

Draw a face in three passes: first only straight and curved lines; then turn lines into eyes, nose, mouth; then arrange those into a face. You just worked the way a deep network does — simple to complex.

You can now explain Deep Learning.

THE INTUITION BEHIND IT

A crowd of simple parts can do what no single part could — you have seen it in ant trails and assembly lines. Here the parts are the neurons of Chapter 9, stacked deep, with weighted threads between them.

WHERE THIS GOES NEXT

Make these stacks vast — billions of weights — and surprising new abilities appear. Book 3 calls it scaling.

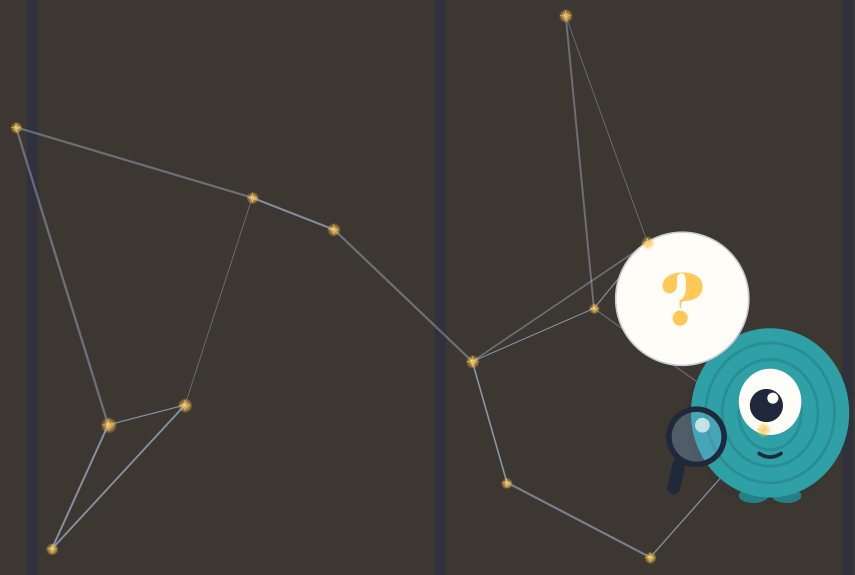
TALK IT OVER

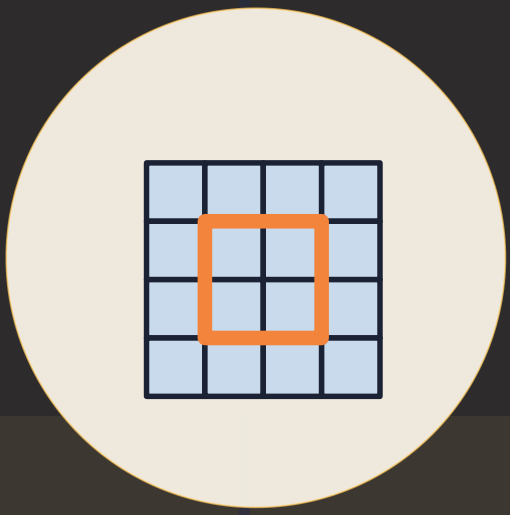
If even the people who built a deep network can't fully explain its decisions, how much should we trust it with important ones?

11

Machines That See

A phone finds a face whether it's top-left, bottom-right, near or far. How does a machine spot the same thing *anywhere* in a picture?





A face in the corner is, in pixel-numbers, totally different from the same face in the centre. Checking every spot separately would take forever. The fix is a clever piece of laziness that powers nearly all machine vision.

YOU ALREADY KNOW THIS

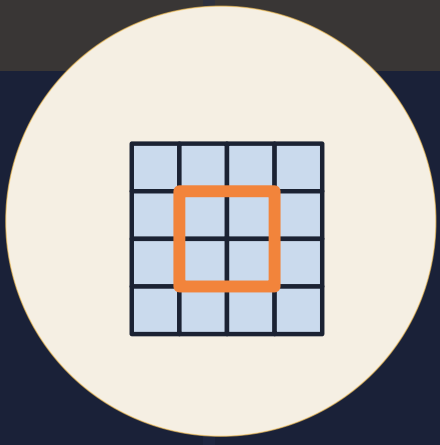
Before the machinery, three places you have already met the idea —

Cut a small window in cardboard and slide it across a picture, looking for one thing — say, a vertical edge. The same little detector, used everywhere.

Your eye does something like this: you don't memorise “nose at position 47”. You have a nose-detector that works wherever the nose happens to be.

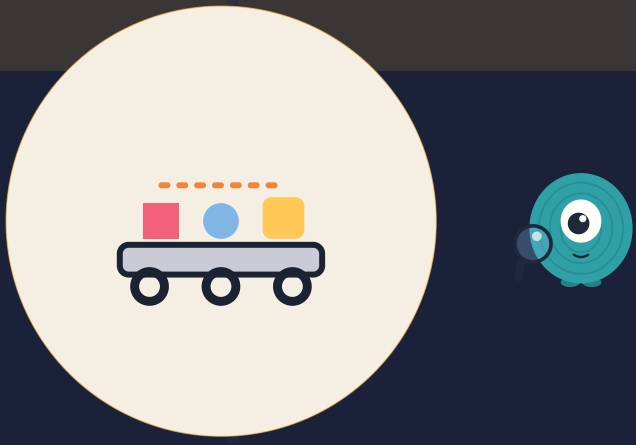
Find edges anywhere, then notice where edges meet to make corners, then where corners make shapes — each step reusing the step below.

Make a small detector



Build a tiny pattern-finder — a **filter** — that fires when it sees one feature, like an edge.

Slide it everywhere



Drag that same filter across the whole image. Wherever the feature appears, it lights up. Reusing one detector across every position is **convolution**.

HOW IT ACTUALLY WORKS
STEP 3 OF 3

Stack the layers

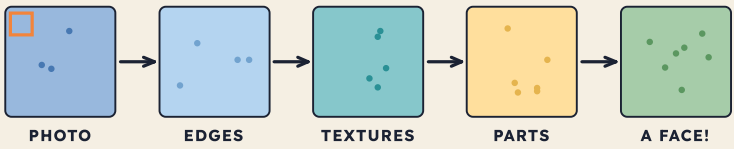
Early filters find edges; the next layer combines them into textures; the next into parts; the last into whole objects. Edges → textures → parts → “a cat”.

Sliding one detector across an image to find a feature anywhere has a name:

CONVOLUTION

Convolution — how machines learned to see.

one detector slides across the whole image



edges → textures → parts → an object

Your mental model. An image passing through filter layers: edges, then textures, then parts, then an object.

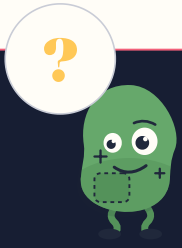
WHERE IT BREAKS

These machines lean on *texture* more than shape, so an odd angle, strange lighting, or a weird background can fool them. A cow on a beach may be missed because cows "should" be on grass. They see patterns of pixels, not the world those pixels show.

WATCH OUT

Myth: "It sees the way we do."

Truth: It detects statistical patterns of pixels. It has no idea what a cat *is* — only what cat-pixels tend to look like, which is why small tricks can break it.



Milo looks for where it cracks — because knowing the limits is how you use it well.

GO DEEPER · WEIGHT-SHARING IS THE TRICK

The reason convolution is so efficient is **weight-sharing**: instead of learning a separate detector for every position, the network learns one and reuses it everywhere. Far fewer weights, and the feature is found no matter where it sits.

TRY THIS

Cut a 1-cm square hole in paper. Slide it across a photo looking only for edges. Notice you find edges all over with one little window — no need for a different eye at every spot.

You can now explain Convolution.

THE INTUITION BEHIND IT

Choosing a few features to notice (Chapter 1) gets a twist here: instead of looking in one spot, convolution hunts for the same feature everywhere in an image at once — the way your eye finds a face wherever it appears.

WHERE THIS GOES NEXT

Join vision with language and you get models that see *and* read together — multimodal AI, in Book 3.

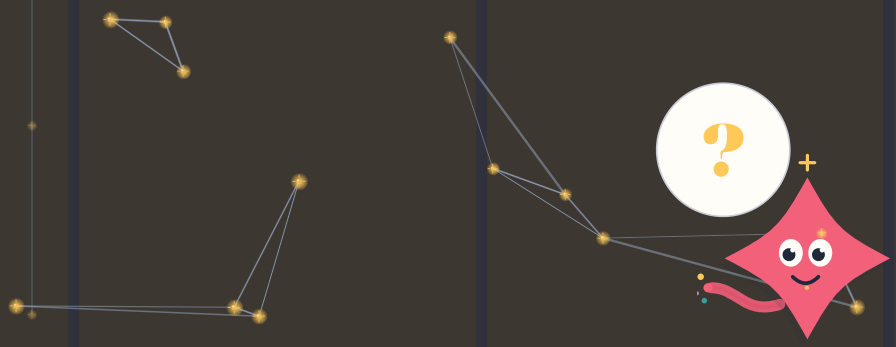
TALK IT OVER

If a vision model fails the moment a familiar object appears in an unfamiliar place, did it ever really “recognise” the object?

12

Machines That Focus

In a long sentence, the word “it” might point back ten words. How does a machine know *which* earlier word matters right now?



Reading isn't just marching word by word. To understand “the trophy didn't fit in the case because *it* was too big”, you have to look *back* at the right word. Machines needed a way to look back too — and it changed everything.

YOU ALREADY KNOW THIS

Before the machinery, three places you have already met the idea —

In a dark room full of objects, you don't light everything equally. You swing a spotlight onto the one that matters now, and dim the rest.

Re-reading a hard sentence, your eyes jump back to the key word — the name, the “it”, the “because” — and weigh it more heavily.

Answering a question, you mentally underline the words that count and skim past the filler. You *weight* what to focus on.

Score every other piece

For the word it's working on, the machine asks how relevant *every* other word is — giving each a score.

Turn scores into focus

High scores become a bright spotlight, low scores stay dim. The machine then mostly listens to the words it scored highly.

Mix the focused-on meanings

It blends the meanings of the words it's focusing on into its understanding of the current word. This score-and-focus move is **attention**.

Weighing how much every piece matters and focusing there has a name:

ATTENTION

Attention — the focus that powers modern AI.

"fast" looks back — most at "cat" and "ran"

attention = how much each word weighs the others

Your mental model. A sentence with glowing links: "fast" attends most strongly back to "cat" and "ran".

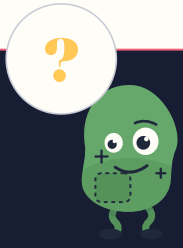
WHERE IT BREAKS

Attention shows you *where* the model looked — not *why* its answer is right. A model can spotlight exactly the sensible words and still conclude something false. Looking at the right place is not the same as understanding it.

WATCH OUT

Myth: "Attention equals understanding."

Truth: Attention is a focusing mechanism, not comprehension. It's a powerful way to weigh context — but a spotlight on the right word doesn't guarantee the right thought.



Milo looks for where it cracks — because knowing the limits is how you use it well.

GO DEEPER · A SOFT, LEARNED LOOK-UP

Attention works like asking a question and matching it against labels to fetch the best answers — but *softly*, blending several matches by how well they fit, and the machine *learns* what makes a good match. It's a look-up table that trained itself.

TRY THIS

Read this sentence aloud: “The keys are on the table because they were heavy.” What does “they” point to — keys or table? Notice your mind spotlight one word. That jump is attention.

You can now explain Attention.

THE INTUITION BEHIND IT

The weighting from the neuron in Chapter 9 was fixed. Attention makes the weights *change for every word* — focus that moves.

WHERE THIS GOES NEXT

Stack attention into the architecture called the transformer and you have the engine behind modern AI — Book 3 opens there.

TALK IT OVER

If we can see exactly where a model focused but still can't tell whether it understood, how much does that "explanation" really explain?

PART FOUR

The Loom & the Kiln

Language and generation

YOUR FOUR GUIDES

Pixel
the Apprentice — asks the sharp question

Nova
the Maker — works the Loom & Kiln

Echo
the Analyst — reads the dials

Milo
the Tester — breaks things to learn

The Word-Loom weaves one token at a time into sentences; the Kiln forms images out of noise. How generation works — and why a confident answer can still be wrong.

13

Guessing the Next Word

A machine that does nothing but guess the next word can write an essay, a poem, even code. How can so simple a trick do so much?

on the

?

It feels like there must be a tiny author in there, planning paragraphs. There isn't. There is one humble move, repeated — and the surprise of this chapter is just how far that single move can go.

YOU ALREADY KNOW THIS

Before the machinery, three places you have already met the idea —

A friend starts “better late than…” and you blurt “never” before they finish. You predicted the next word without trying.

Your phone's autocomplete suggests the next word from the ones before it. Tap, tap, tap and it writes a whole message — one guess at a time.

A bard telling a tale builds it word by word, each new word leaning on everything already said.

Predict one token

Given the tokens so far (Chapter 3), the machine produces a probability for every possible next token — how likely each is to come next.

Pick and append

It chooses one token, adds it to the text, and now has a slightly longer prompt. A model that does this is a **language model**.

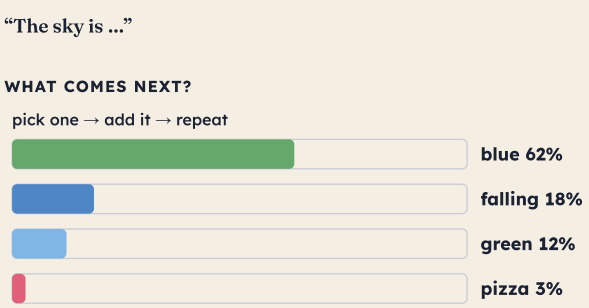
Feed it back and repeat

The longer text becomes the new input, and it predicts again. Word by word, an essay appears — built from one move done thousands of times. The recent text it can use is its **context window**.

A machine that predicts the next token over and over has a name:

LANGUAGE MODEL

A language model — fluent by guessing, one token at a time.



the recent text it can use = the context window

Your mental model. Tokens in → probabilities over the next token → pick one → append → repeat.

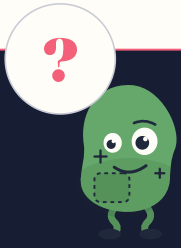
WHERE IT BREAKS

It has no memory beyond its context window — push past that and the start simply falls away, forgotten. And being *fluent* is not being *knowledgeable*: it's choosing plausible words, not checking facts. Smoothness and truth are two different things.

WATCH OUT

Myth: "It looks facts up in a database."

Truth: It stores no database of facts to query. It predicts likely tokens from patterns it absorbed in training, which is why it can sound authoritative and still be wrong.



Milo looks for where it cracks — because knowing the limits is how you use it well.

GO DEEPER · TEMPERATURE = HOW ADVENTUROUS

The machine doesn't always pick the single likeliest token. A setting called **temperature** controls the gamble: low temperature plays safe and predictable, high temperature takes risks and gets creative — or weird. Same model, different daring.

TRY THIS

Play one-word-each storytelling with friends, going round the circle. Notice nobody planned the ending — yet a story appeared, each word predicted from the last. That's a language model, made of people.

You can now explain Language Model.

THE INTUITION BEHIND IT

You finish other people's sentences without thinking — the next word is just that likely. A language model does only this, on the tokens of Chapter 3, using the attention of Chapter 12 to weigh what came before.

WHERE THIS GOES NEXT

Give a model tools and a way to fetch real documents — retrieval — and the next-word trick gets far more reliable. Book 3.

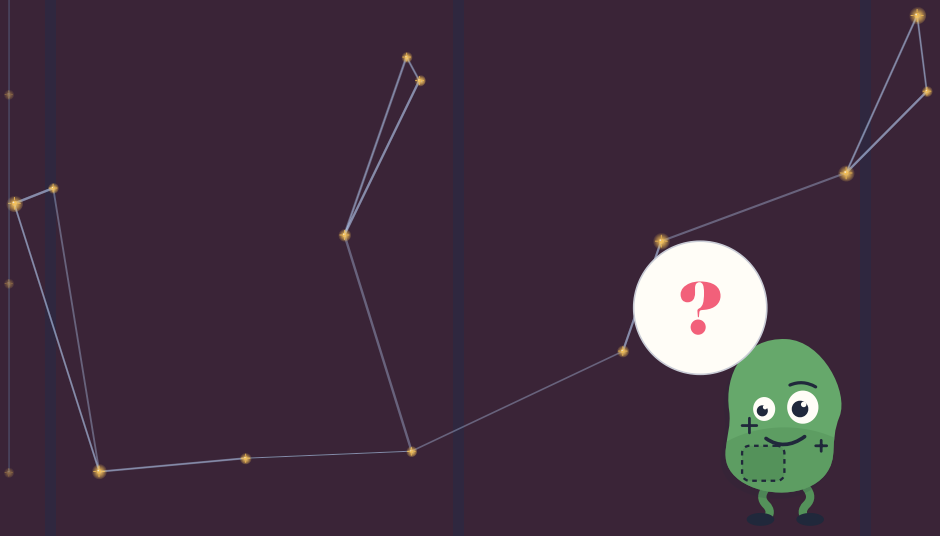
TALK IT OVER

If a system writes flawless, confident prose with no way to check whether it's true, where should a careful reader put their trust?

14

Why It Sounds So Sure (and Is Wrong)

Ask for a fact it doesn't know and a model won't shrug — it invents a smooth, confident, completely false answer. Why does it lie so fluently?



This isn't a glitch bolted on by accident. It's the next-word machine from Chapter 13 doing exactly what it was built to do. Understand that, and these confident mistakes stop being mysterious.

YOU ALREADY KNOW THIS

Before the machinery, three places you have already met the idea —

A smooth talker, asked something they don't know, doesn't pause — they fill the gap with whatever *sounds* right, delivered with total confidence.

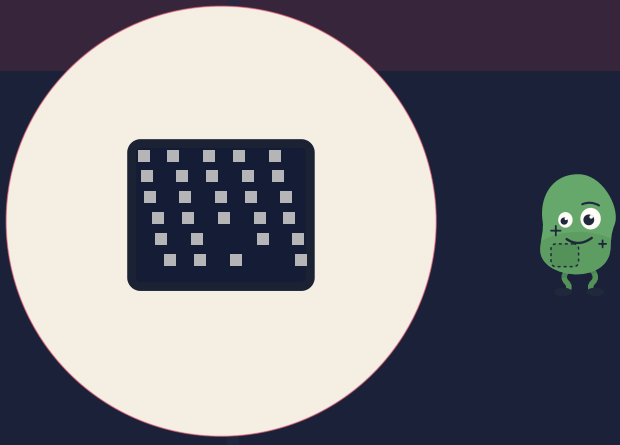
You misremember a story so vividly that it feels true. Your mind produced a plausible memory, not a checked one.

Stuck on an exam question, you write a confident-sounding answer that flows nicely — even though you're guessing.

It scores plausibility, not truth

Remember: the model picks the most *likely-sounding* next words, not the most *true* ones. It has no separate fact-checker inside.

Gaps get filled smoothly



When it never reliably learned a fact, the gap doesn't stay blank. The most fluent continuation wins — so a believable falsehood gets chosen over an honest “I don't know”.

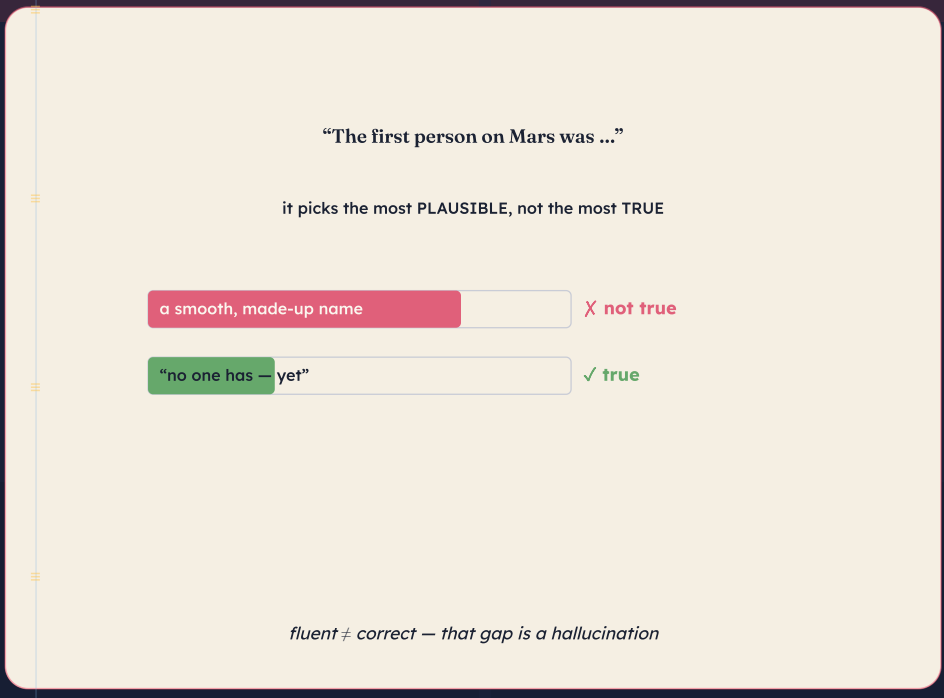
Confidence is just fluency

The smooth, certain tone isn't evidence of knowledge. It's the same machinery that makes true answers flow. A made-up answer called a **hallucination** sounds exactly as sure as a real one.

A confident, fluent, false answer has a name:

HALLUCINATION

A hallucination — prediction working as designed, missing the facts.



Your mental model. Two next-word options: the fluent-but-false answer scores higher than the honest one.

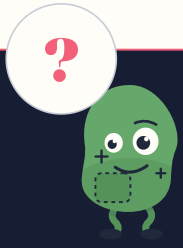
WHERE IT BREAKS

Because fluency hides error, you often can't tell a hallucination from a fact by reading it — both sound polished. The danger is worst for rare, specific facts the model barely saw, which are exactly the ones people most want to look up.

WATCH OUT

Myth: "If it sounds sure, it's right."

Truth: Confidence and correctness are unrelated here. A model is equally fluent when right and when wrong — so tone tells you nothing. Check, don't trust the polish.



Milo looks for where it cracks — because knowing the limits is how you use it well.

GO DEEPER · WHY RARE FACTS ARE THE MOST FRAGILE

A model learns common patterns strongly and rare ones faintly. So a famous date it has "seen" a million times is fairly safe, while an obscure name it met once or never is where it confidently invents. The rarer the fact, the thinner the ice.

TRY THIS

Ask a friend to confidently "explain" a made-up word (say, *florbing*) for ten seconds without pausing. Notice how convincing fluent nonsense can be — then imagine it in your own voice, on a screen.

You can now explain Hallucination.

THE INTUITION BEHIND IT

A smooth talker who never says “I don't know” will confidently fill any gap — and so will the next-word machine from Chapter 13. That is all a hallucination is: plausibility winning over truth.

WHERE THIS GOES NEXT

Grounding answers in real sources and verifying them is the active frontier — Book 3's reliability story.

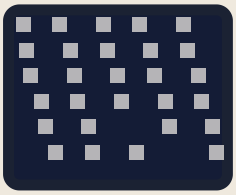
TALK IT OVER

If a machine literally cannot tell when it's making something up, whose job is it to catch the error before it spreads?

15

Painting from Noise

Type “a fox in a spacesuit” and a machine paints it in seconds — a picture that never existed. Where does it come from? Not from a search.



It isn't pasting clippings. The Kiln makes images by a method that sounds backwards at first: it starts with pure mess and *removes* its way to a picture, like a sculptor freeing a shape from stone.

YOU ALREADY KNOW THIS

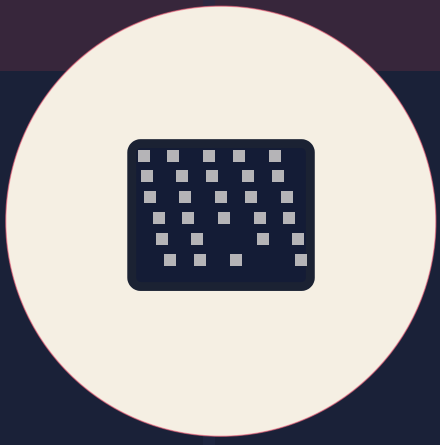
Before the machinery, three places you have already met the idea —

Stare at TV snow and your brain insists it sees shapes — faces, animals — in the random dots. Order half-glimpsed inside noise.

A sculptor doesn't add a statue to a block of marble. They *remove* everything that isn't the statue. Creation by taking away.

A photographer in a darkroom watches an image slowly surface out of a blank grey sheet — faint, then clearer, then sharp.

Start from pure noise



Begin with a square of random static — no picture at all, just dots.

Remove a little noise

Step by step, the machine predicts what noise to *subtract* to move the mess slightly toward a real image — guided at every step by your words.

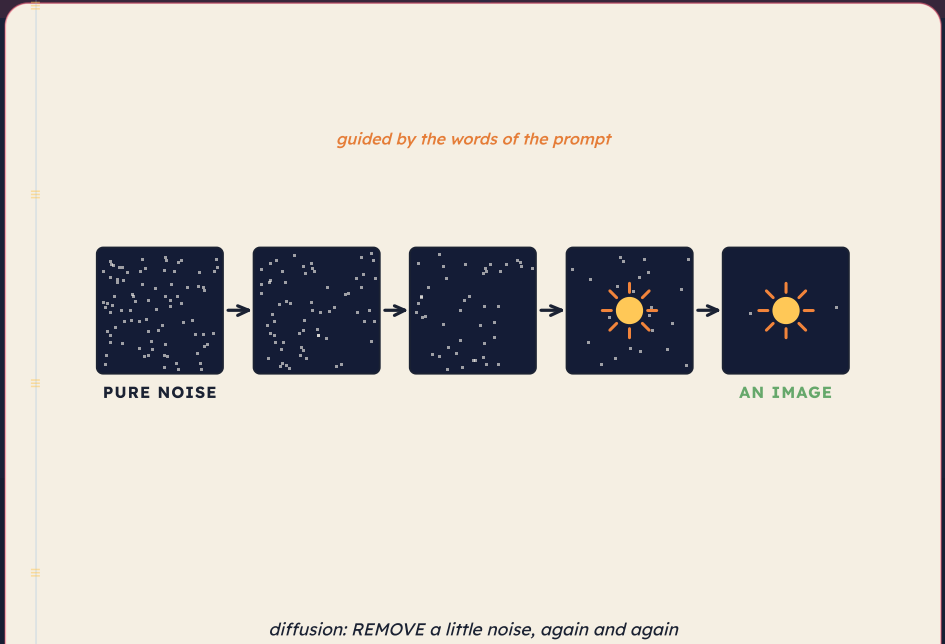
Repeat until it's clear

Denoise again and again. The fog lifts a little each time until a sharp picture matching the prompt remains. This denoising method is **diffusion**.

Making an image by removing noise step by step has a name:

DIFFUSION

Diffusion — painting by taking the mess away.



Your mental model. A strip from pure noise → blur → image, each step guided by the words of the prompt.

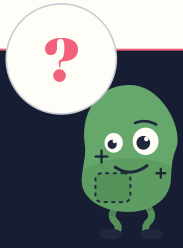
WHERE IT BREAKS

It blends patterns; it doesn't *know* things. So hands sprout extra fingers, written text comes out as gibberish, and physics quietly breaks — because the machine learned what pixels usually look like, not how hands or letters or gravity actually work.

WATCH OUT

Myth: "It copies a picture from its memory."

Truth: It stores no gallery to copy from. It generates fresh pixels by denoising, shaped by patterns across millions of images — which is why it can make things that never existed.



Milo looks for where it cracks — because knowing the limits is how you use it well.

GO DEEPER · WHY LEARNING TO DENOISE TEACHES DRAWING

In training, the machine is shown real images with noise added, and learns to undo it. To remove noise well it must learn what real images are *like* — edges, shapes, how a face is arranged. Master un-noising and you've secretly mastered generating.

TRY THIS

Squint at a cloud or a stain until you “see” an animal in it. Your brain just pulled a shape out of noise — the same move, in spirit, that diffusion makes thousands of times.

You can now explain Diffusion.

THE INTUITION BEHIND IT

Making something new is really recombining what you know — and a sculptor frees a statue by removing, not adding. Diffusion does both: it carves an image out of pure noise, guided by your words.

WHERE THIS GOES NEXT

Push this toward models that simulate whole scenes and physics — world models — and you reach Book 3.

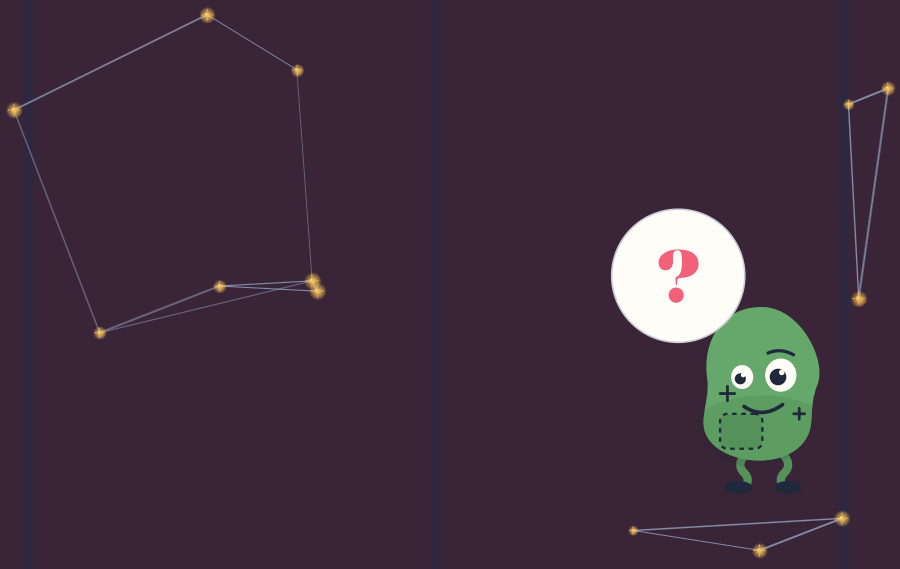
TALK IT OVER

If an image-maker has never seen a real hand, only pictures of hands, why should we expect it to get the fingers right?

16

Forger and Detective

Can two machines, with no teacher, make each other better just by competing? Set a forger against a detective and watch what happens.



Most learning needs examples with answers. But there's a sly way to get realism with none: pit two machines against each other so that beating the rival is the whole lesson. Their contest does the teaching.

YOU ALREADY KNOW THIS

Before the machinery, three places you have already met the idea —

A forger paints fake masterpieces; a detective tries to spot them. Every time the detective catches a fake, the forger learns to paint better.

Every time a fake slips through, the detective studies harder. Each one sharpens the other, round after round.

Two chess rivals of equal skill: neither has a coach, yet both climb because each game exposes the other's weakness.

HOW IT ACTUALLY WORKS
STEP 1 OF 3

One machine makes fakes

A **generator** produces fake examples — images, say — trying to pass them off as real.

Another judges them

A **discriminator** sees real and fake mixed together and tries to tell which is which, scoring each.

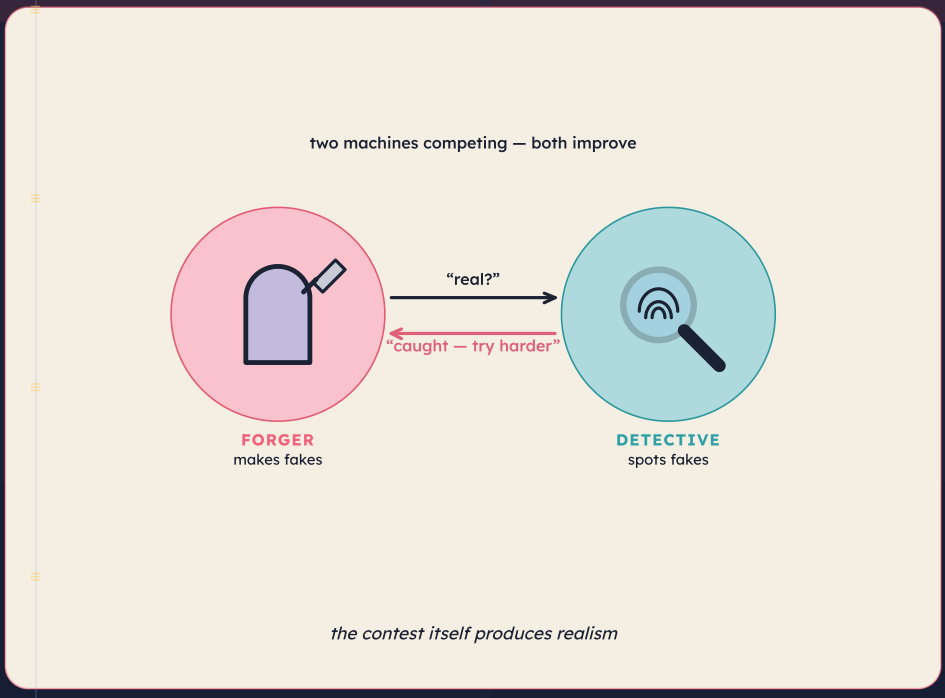
They train each other

The generator improves to fool the judge; the judge improves to catch the fakes. This duelling pair is a **GAN** — and their contest grinds out startling realism.

Two machines that improve by competing have a name:

GAN

A GAN — a forger and a detective, teaching each other.



Your mental model. The duel loop: the forger makes fakes, the detective scores real-vs-fake, both improve.

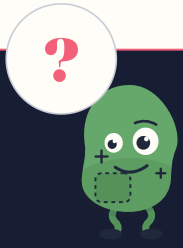
WHERE IT BREAKS

The duel is famously unstable — one side can overpower the other and learning collapses. A common failure is **mode collapse**: the generator discovers one fake that fools the judge and just makes *that*, over and over, losing all variety.

WATCH OUT

Myth: "GANs are how all AI images are made now."

Truth: Not anymore. Diffusion (Chapter 15) has largely taken over image generation. GANs were a huge step and still matter — but the field moved on.



Milo looks for where it cracks — because knowing the limits is how you use it well.

GO DEEPER · GANS VS DIFFUSION, BRIEFLY

A GAN makes an image in one bold shot and is judged whole; diffusion sculpts an image gradually by denoising. Diffusion trains more stably and gives finer control, which is why it now dominates — but the GAN's idea of "learning by competition" echoes everywhere.

TRY THIS

Play "spot the lie": a friend tells two true stories and one false, trying to make the false one sound real; you try to catch it. After a few rounds, both of you get sharper. That's a GAN at a sleepover.

You can now explain Gan.

THE INTUITION BEHIND IT

Two evenly-matched rivals push each other higher than either could climb alone. A GAN turns that into a way to create — a forger and a detective improving each other — a cousin of the diffusion idea from Chapter 15.

WHERE THIS GOES NEXT

When forgers get this good, telling real from fake — deepfakes and provenance — becomes everyone's problem. That's Chapter 23, and Book 3.

TALK IT OVER

If a machine can generate fakes good enough to fool the best detector, what happens to our old habit of believing what we see?

PART FIVE

The Proving Grounds

How well does it really work?

YOUR FOUR GUIDES

Pixel
the Apprentice — asks the sharp question

Nova
the Maker — works the Loom & Kiln

Echo
the Analyst — reads the dials

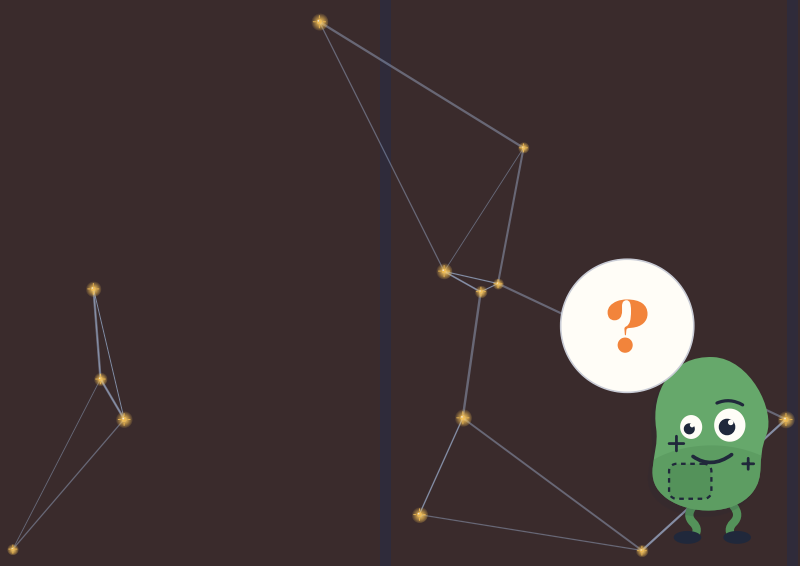
Milo
the Tester — breaks things to learn

Test ranges, dials, and tricksters. Where the mechanism breaks: misleading scores, false confidence, tiny attacks, and a world that keeps moving.

17

How Right Is "Right"?

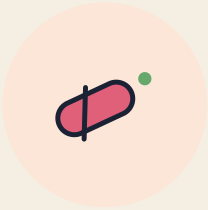
A test that is "99% accurate" can be worse than useless. How can being right almost all the time still be a disaster?



In the Proving Grounds we stop asking “does it work?” and start asking “how, exactly, does it fail?” And the first surprise is that a single, impressive-sounding number can hide everything that matters.

YOU ALREADY KNOW THIS

Before the machinery, three places you have already met the idea —



A disease strikes 1 person in 1000. A test that simply says “healthy” to everyone is 99.9% accurate — and catches nobody. The headline number lied.

Crying wolf: a false alarm wastes everyone's time; a missed real wolf costs the flock. The two mistakes are not the same size.

A face-unlock that wrongly lets a stranger in is a very different failure from one that wrongly keeps *you* out — even at the same “accuracy”.

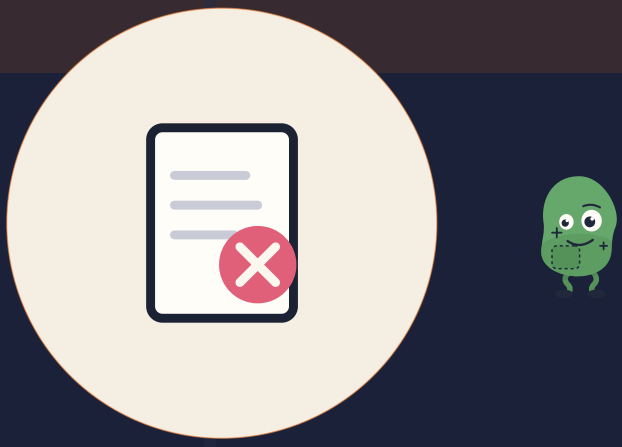
Sort every outcome

There are four cases: correctly flagged, correctly cleared, a **false positive** (false alarm), and a **false negative** (a miss). Lay them in a grid.

Weigh the two mistakes

A false alarm and a miss usually cost very different amounts. Which one you can least afford depends entirely on the job — medicine, spam, security.

Don't trust one number

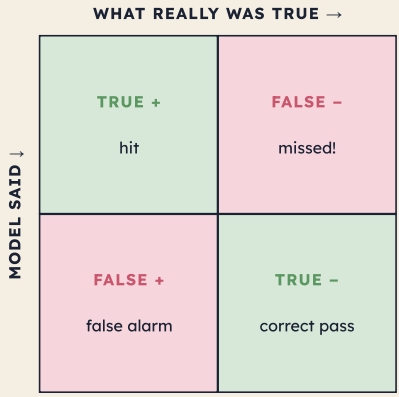


Benchmarks — standard test sets that score and rank models — are useful but gameable, and a single accuracy score hides which kind of mistake the model makes.

The different ways a model can be wrong have names:

ERROR TYPES

False positives and false negatives — a false alarm and a miss are not the same mistake.



"99% accurate" can still miss the rare case that matters

Your mental model. A 2×2 confusion grid: hits, misses, false alarms, and correct passes are four different things.

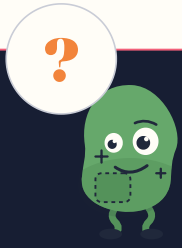
WHERE IT BREAKS

Any single score throws away the shape of the failures. Worse, when a benchmark's answers leak into training, models "ace" it while learning nothing — so a sky-high score can mean the test was beaten, not the problem solved.

WATCH OUT

Myth: "A high score on a benchmark means it's good in the real world."

Truth: A benchmark is one fixed test under lab conditions. Real life is messier and shifts (Chapter 20). Topping a leaderboard is a clue, not a guarantee.



Milo looks for where it cracks — because knowing the limits is how you use it well.

GO DEEPER · PRECISION VS RECALL, IN PLAIN WORDS

Precision asks: of the things it flagged, how many were truly right? **Recall** asks: of the things it should have flagged, how many did it catch? You can usually buy more of one only by giving up some of the other — so which matters more depends on the cost of each mistake.

TRY THIS

Imagine a sweet-shop alarm that buzzes for shoplifters. List one harm if it buzzes too often, and one harm if it stays quiet too often. Which mistake would you rather it made — and why?

You can now explain Error Types.

THE INTUITION BEHIND IT

A false alarm and a missed warning feel completely different in real life — and they should. Checking a model means looking past one tidy “accuracy” number to ask which kind of mistake it makes, and what that mistake costs.

WHERE THIS GOES NEXT

When the score becomes the goal, people optimise the score instead of the goal — Goodhart's law, a Book 3 theme.

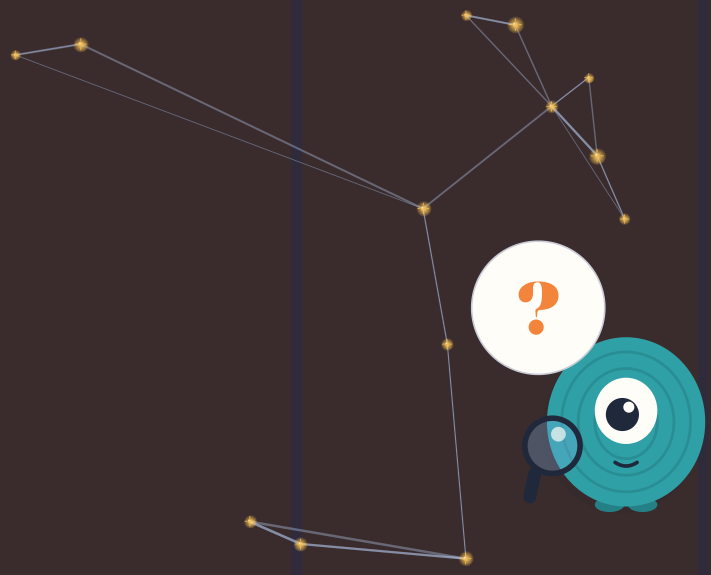
TALK IT OVER

If two models share the same accuracy but fail in opposite ways, can “accuracy” alone ever tell you which one to trust?

18

The Confidence Trick

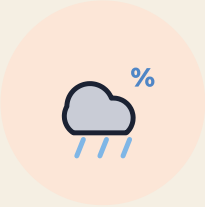
A model says it's “90% sure.” Should you believe the 90? A number that sounds like honesty can be the biggest illusion of all.



We've seen models be confidently wrong (Chapter 14). Now we ask a sharper question: when a machine puts a *number* on its confidence, does that number mean anything? Sometimes it's gold — and sometimes it's theatre.

YOU ALREADY KNOW THIS

Before the machinery, three places you have already met the idea —



A weather app says “70% chance of rain.” If, across all its 70% days, it rains about 70 times in 100, that number is trustworthy. If it never rains, the 70 is meaningless.

An overconfident friend says they're “definitely right” about everything — and is right half the time. Their confidence carries no information.

A fuel gauge that reads “half full” when the tank is nearly empty isn't just wrong — it's *dangerously* wrong, because you trusted it.

HOW IT ACTUALLY WORKS
STEP 1 OF 3

Collect the confident claims

Gather every time the model said “90% sure” and check how often those were actually right.

Compare claim to reality

If its 90%-sure answers are right about 90% of the time, and its 60%-sure ones right about 60%, the model is **well-calibrated** — its numbers mean what they say.

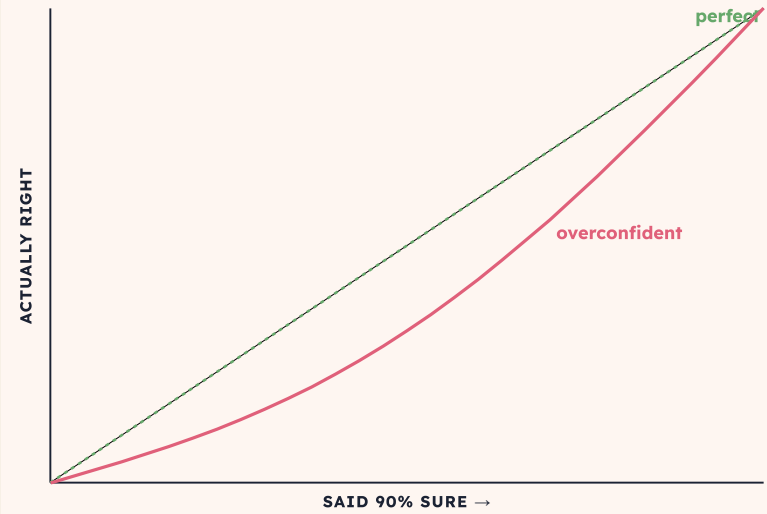
Treat uncertainty as information

A calibrated “I'm only 55% sure” is genuinely useful — it tells you to double-check. “I don't know” can be the most honest, valuable output a machine gives.

When a model's confidence numbers actually match reality, that has a name:

CALIBRATION

Calibration — and uncertainty, honestly stated, is information.



well-calibrated: 90%-sure answers are right ~90% of the time

Your mental model. A calibration curve: the perfect diagonal versus an overconfident model sagging below it.

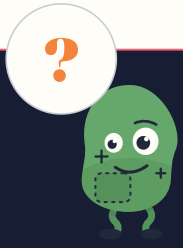
WHERE IT BREAKS

Most raw models are *over*confident — their 90% should really be 70%. Worse, a model can be well-calibrated on its training world and wildly off the moment the world shifts (Chapter 20), so yesterday's honest numbers can quietly go bad.

WATCH OUT

Myth: "The percentage is the truth."

Truth: The percentage is a *claim* that itself needs checking. Only a calibrated model's numbers can be taken at face value — and even then, only inside the world it was tested on.



Milo looks for where it cracks — because knowing the limits is how you use it well.

GO DEEPER · CALIBRATION IS NOT ACCURACY

A model can be very accurate yet badly calibrated (right often, but its confidence numbers are junk), or not very accurate yet well calibrated (it knows when it's unsure). They measure different virtues: being right, versus knowing how right you are.

TRY THIS

Before a quiz, write your % confidence next to each answer. Afterwards, look at all your "80% sure" answers — were about 80% correct? If you were way off, you just measured your own calibration.

You can now explain Calibration.

THE INTUITION BEHIND IT

A forecast's "70%" is only useful if it rains on about 70% of such days. Calibration asks the same of any machine's confidence — and pairs with the honesty-about-limits theme running through these Proving Grounds.

WHERE THIS GOES NEXT

Honest uncertainty is central to deploying AI safely in medicine, cars, and courts — a Book 3 cornerstone.

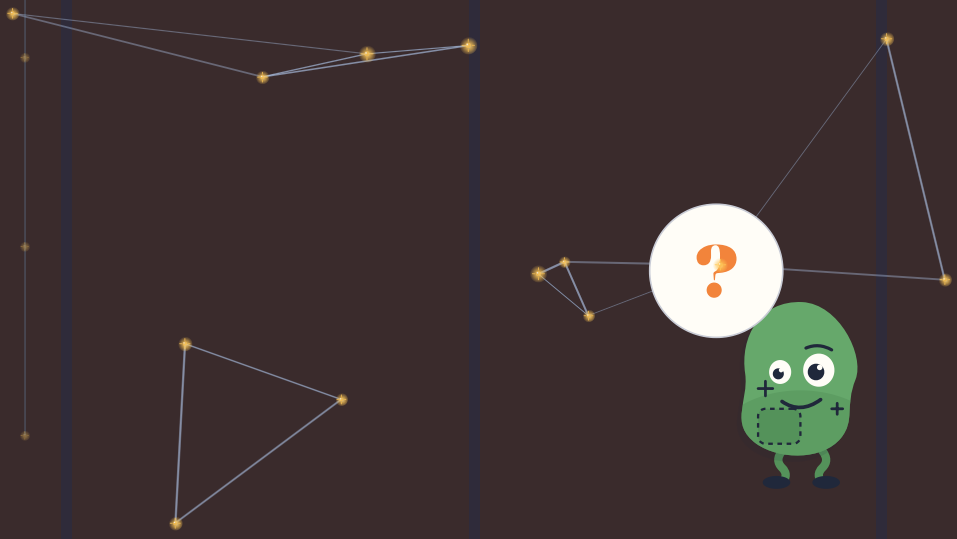
TALK IT OVER

Would you rather have a model that's usually right but never admits doubt, or one a bit less accurate that tells you exactly when to check?

19

Fooling the Machine

Add a sticker a human barely notices, and a model calls a banana a toaster — with 99% confidence. How can something so small break something so smart?



This isn't a random bug. It reveals something deep about how these machines actually compute: they live in a strange high-dimensional space full of invisible trapdoors, and you can aim a tiny push straight at one.

YOU ALREADY KNOW THIS

Before the machinery, three places you have already met the idea —

A few chosen pixels — invisible to you — nudge the model's numbers across a decision boundary (Chapter 4). To you: still a banana. To it: a toaster.

Optical illusions fool *your* eyes through their shortcuts. Machines have their own blind spots, just different ones.

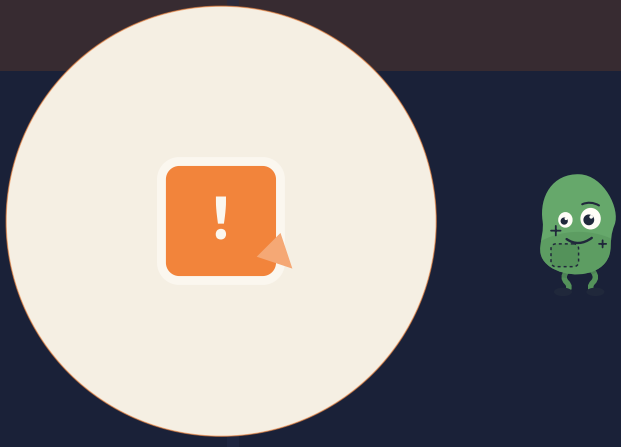
A husky-vs-wolf model that secretly learned “snow = wolf”. Photograph a husky in snow and it cries wolf — it learned the shortcut, not the animal.

HOW IT ACTUALLY WORKS
STEP 1 OF 3

Find the model's blind spot

Because the model is just arithmetic, you can work out exactly which tiny pixel changes move its answer the most — and aim there.

Add an invisible nudge



Apply that minute, calculated perturbation. It's far too small for a human to see, but it shoves the model's numbers over a boundary. This rigged input is an **adversarial example**.

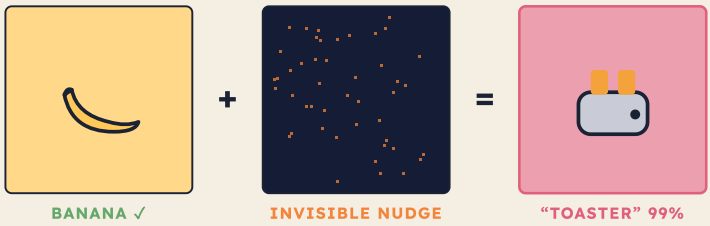
Or exploit a shortcut

Even without an attack, models grab **spurious correlations** — easy cues that happen to work in training (snow, watermarks, backgrounds) but aren't the real thing.

An input crafted to fool a model has a name:

ADVERSARIAL EXAMPLE

An adversarial example — a tiny nudge through an invisible trapdoor.



a tiny change exploits how the model really computes

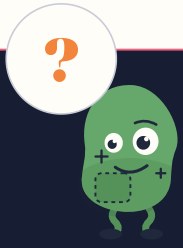
Your mental model. Banana + an invisible, calculated nudge = a confident “toaster”; and a “snow = wolf” shortcut.

Making models robust to these tricks is genuinely hard — patch one hole and attackers find another. And shortcuts are everywhere, because the model will always take the easiest cue that lowers its loss, whether or not it's the cue we meant.

WATCH OUT

Myth: "If it works on photos, it must understand them."

Truth: Working on ordinary photos and being fooled by a sticker live side by side. High accuracy hides these holes; it doesn't fill them.



Milo looks for where it cracks — because knowing the limits is how you use it well.

GO DEEPER · WHY HIGH-DIMENSIONAL SPACE HIDES HOLES

With hundreds of input directions, there are vastly more ways to nudge an image than we could ever check. Almost every real image sits near some unseen direction that flips the answer. The very roominess that makes models powerful also leaves them riddled with trapdoors.

TRY THIS

Find a classic optical illusion online. You *know* the lines are equal, yet your eyes refuse. Now you've felt, from the inside, how a system can be confidently and structurally wrong.

You can now explain Adversarial Example.

THE INTUITION BEHIND IT

In Chapter 11, vision models leaned on texture and background. Adversarial examples and shortcuts are that weakness, weaponised.

WHERE THIS GOES NEXT

Deliberately attacking models to find their failures — red-teaming — is now a core safety practice. More in Book 3.

TALK IT OVER

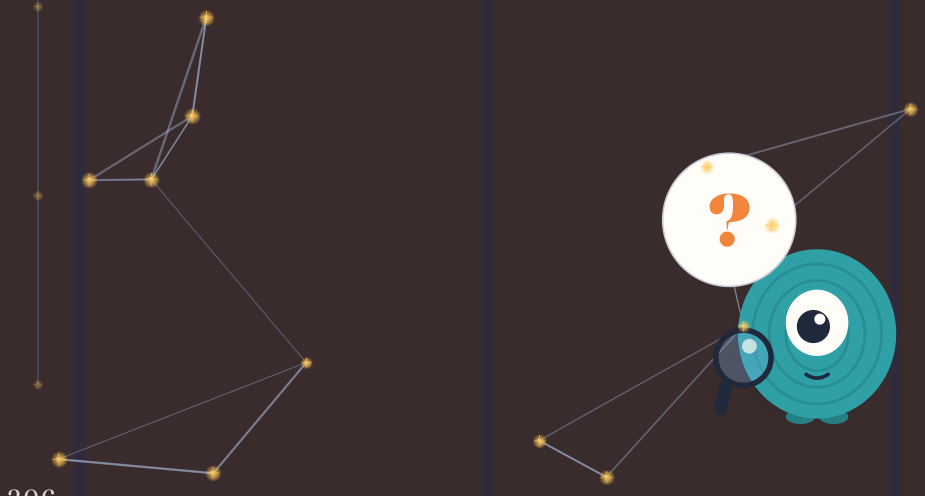
If a model can be flipped by a change no human can even see, what should we demand before trusting it with a car or a diagnosis?

CHAPTER 20 · THE PROVING GROUNDS

When the World Moves

20

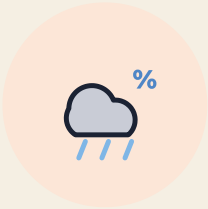
A model that's brilliant in the lab fails on launch day — without anyone changing a line of it. What went wrong while it sat still?



Nothing about the model changed. The *world* changed. This quiet failure is one of the most common ways real AI breaks, and it ties together everything in the Proving Grounds.

YOU ALREADY KNOW THIS

Before the machinery, three places you have already met the idea —



A self-driving demo that aced sunny test-tracks meets its first heavy snow on launch day — conditions it simply never trained on.

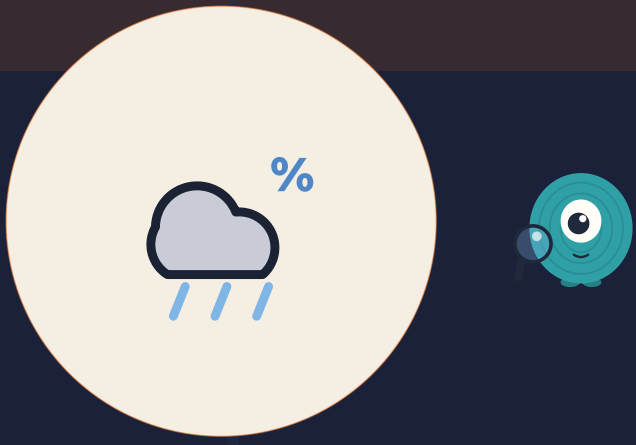
A slang word that meant one thing last year flips meaning this year. A model trained on the old texts reads it exactly wrong.

A medical model trained on one hospital's scanners is deployed to another with different machines and lighting — and its accuracy quietly sinks.

Training captures one slice of the world

A model only ever sees its training data — one particular cloud of examples from one time and place.

The world drifts away from it



At use-time, the real inputs come from a different cloud: new weather, new slang, new cameras. The two clouds no longer line up.

HOW IT ACTUALLY WORKS
STEP 3 OF 3

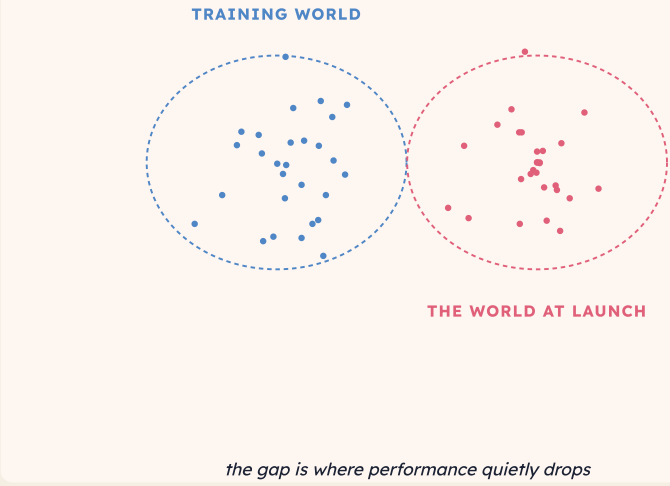
Performance silently degrades

Where the clouds don't overlap, the model is guessing outside what it learned — and it usually keeps sounding confident while getting worse. This gap is **distribution shift**.

When the world at use-time drifts from the training world, that has a name:

DISTRIBUTION SHIFT

Distribution shift — and the model rarely knows it has happened.



the gap is where performance quietly drops

Your mental model. The training-data cloud beside the deployment cloud — the shaded gap is where it fails.

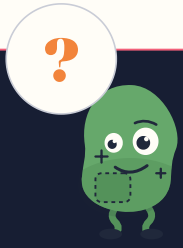
WHERE IT BREAKS

The cruel part: models rarely know when they've left their world. They don't raise a hand and say “this is unfamiliar” — they answer anyway, with the same confidence (which is why calibration, Chapter 18, matters so much).

WATCH OUT

Myth: “Tested once means safe forever.”

Truth: A model is only validated for the world it was tested in. As reality drifts, that validation expires — so monitoring has to continue after launch, not stop at it.



Milo looks for where it cracks — because knowing the limits is how you use it well.

GO DEEPER · MONITORING AND DRIFT

Because shift is gradual and silent, teams must **monitor** a live model — watching whether today's inputs still look like the training data, and whether accuracy is slipping — so they can retrain before a quiet drop becomes a public failure.

TRY THIS

Learn a game's rules with friends, then have someone secretly change one rule mid-game. Watch your confident moves suddenly misfire. You just felt distribution shift — your "training" no longer matched the world.

You can now explain Distribution Shift.

THE INTUITION BEHIND IT

In Chapter 7, overfitting meant failing on new *examples*. Distribution shift is failing on a new *world* — the same lesson, larger.

WHERE THIS GOES NEXT

Models that keep learning after launch — continual learning, with constant evaluation — are a major Book 3 frontier.

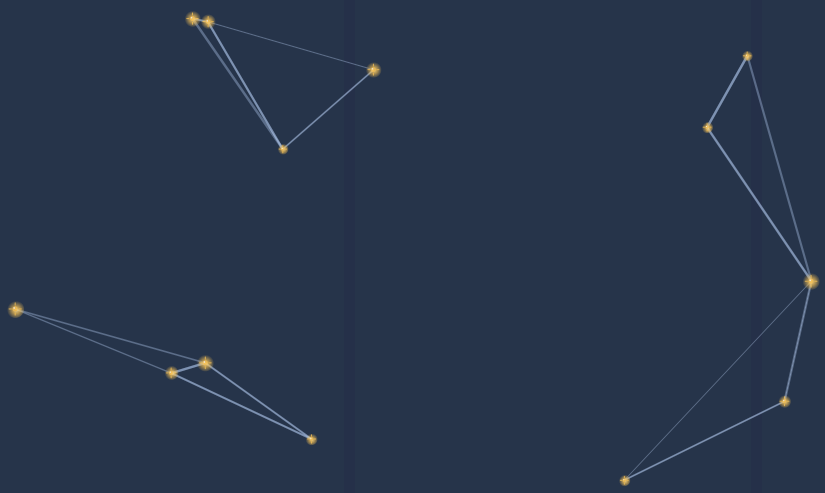
TALK IT OVER

If a model can't tell that the world has moved on, who is responsible for noticing — and how often should we check?

PART SIX

The Commons

Data, people, and choices



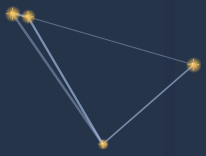
YOUR FOUR GUIDES

Pixel
the Apprentice — asks the sharp question

Nova
the Maker — works the Loom & Kiln

Echo
the Analyst — reads the dials

Milo
the Tester — breaks things to learn



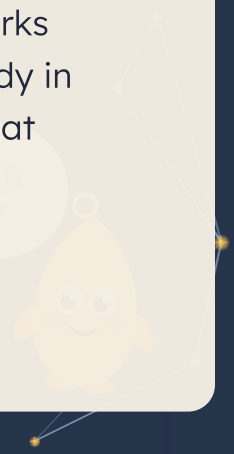
The town outside the Works, where the data comes from and people live with the results. Provenance, bias, privacy, and who is responsible when a machine advises.

21

Where Data Comes From

A model "taught itself" from the internet, people say. But who really did the teaching — and did anyone ask?

Every machine in this book learned from data. In the Commons we step outside the Works to ask the uncomfortable question nobody in the machine can answer: where did all that data actually come from?



YOU ALREADY KNOW THIS

Before the machinery, three places you have already met the idea —

Behind “the AI learned it” are real people — often paid very little — who labelled millions of images and answers by hand, one at a time.

Much training data was *scraped*: copied in bulk from the open web — articles, photos, code — without each author being asked.

A picture of *you*, posted years ago, may sit inside a dataset you never heard of, training models you'll never meet.

STEP 1 OF 3

Data is built, not found

A **dataset** doesn't grow on trees. It's assembled — collected, cleaned, and labelled — by human labour and mass collection, with countless small choices along the way.

STEP 2 OF 3

Trace where it came from

Provenance means the origin and history of the data: who made it, how it was gathered, whether there was consent. It's often murky or undocumented.

Origin shapes everything

Whatever is in that data — its gaps, its errors, its viewpoints — flows straight into the model and out into its answers. Garbage or gold, it all comes downstream.

The origin and history of a model's data has a name:

PROVENANCE

Provenance — where the data came from shapes all it becomes.

where the data came from shapes everything after

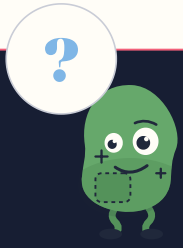
Your mental model. A pipeline from the web and people → a built dataset → the model.

So much is hidden: the labour is invisible, consent is patchy or absent, and the quality is frequently unknown even to the people using the data. You often can't fully audit what a model learned from, which makes its blind spots hard to predict.

WATCH OUT

Myth: "The model learned all by itself."

Truth: It learned from data that people made, collected, and labelled. Every model stands on a great deal of human work — much of it unseen and uncredited.



Milo looks for where it cracks — because knowing the limits is how you use it well.

TRY THIS

List five things you've posted or that others posted of you online. For each, ask: could this be in a dataset? Did anyone ask you? Notice how quickly “my data” gets blurry.

You can now explain Provenance.

THE INTUITION BEHIND IT

Every machine in this book learned from a pile of examples — its data. The grown-up question the Commons asks is the one the machine cannot: whose examples were they, and how were they gathered?

WHERE THIS GOES NEXT

Who owns training data, and whether scraping is fair, are live legal battles — Book 3 walks into the courtroom.

TALK IT OVER

If a model's abilities rest on millions of people's unpaid, unasked-for work, what do its makers owe those people?

22

Bias, Mechanically

How does a “neutral” machine, with no opinions and no feelings, end up treating people unfairly? There's no villain inside — so where does the unfairness come from?

It's tempting to think a machine must be fair because it's “just maths”. But unfairness doesn't need a villain. It seeps in through the mechanism itself, at points we can name precisely.

YOU ALREADY KNOW THIS

Before the machinery, three places you have already met the idea —

A hiring tool trained on a company's past hires learns to copy the past — including who was *not* hired before. It mistakes "what was" for "what's good".

A camera tuned mostly on lighter faces misses darker ones — not from malice, but because its training data was lopsided.

A voice assistant trained mostly on one accent struggles with others, leaving some people unheard.

STEP 1 OF 3

Bias enters through the data

If the examples are skewed — some groups over-represented, others missing — the model faithfully learns the skew. Picture a jar filled only with stones from one street — every one the same colour.

STEP 2 OF 3

And through every choice

It also enters through the *labels* people assign, the *features* we pick, and the **objective** we tell it to optimise. Each is a human choice the machine inherits.

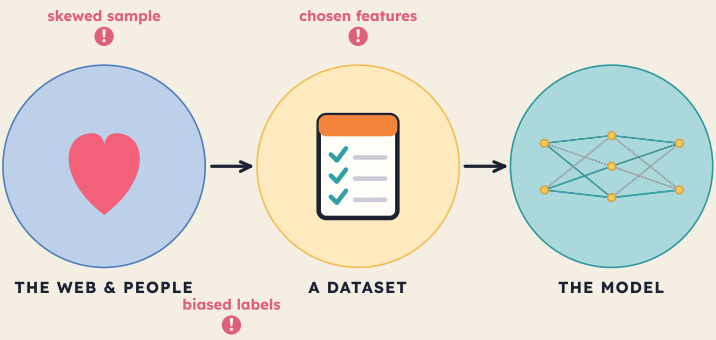
Then it gets amplified

A model doesn't just copy **bias** — chasing low loss, it can *sharpen* a small imbalance into a confident, automated, large-scale one.

Unfairness baked in through data and choices has a name:

BIAS

Bias — it enters mechanically, then gets amplified.



bias enters at every step — then gets amplified

Your mental model. The data pipeline again — now with bias-entry points flagged: sample, labels, features, objective.

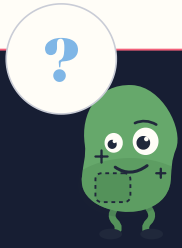
WHERE IT BREAKS

"Debiasing" is contested and only ever partial. Fix one measure of fairness and you can worsen another; the very definition of "fair" differs between people and situations. There's no single switch that makes a model just.

WATCH OUT

Myth: "More data fixes bias."

Truth: More of the *same* skewed data deepens the bias. What helps is more *representative* data — and careful choices — not simply more.



Milo looks for where it cracks — because knowing the limits is how you use it well.

TRY THIS

Ask only your closest friends “what's the best music?” and write the “answer”. Now imagine a model trained only on that. Whose taste became “the truth”? Whose was missing entirely?

You can now explain Bias.

THE INTUITION BEHIND IT

A jar of stones all scooped from one street will teach you something false about the whole world. Bias is that lopsided jar made mechanical — and the pipeline from Chapter 21 shows the exact points where the lopsidedness gets in.

WHERE THIS GOES NEXT

How to define and govern fairness — across cultures and laws — is one of AI's hardest open questions. Book 3.

TALK IT OVER

If different people define “fair” in ways that can't all be satisfied at once, who should decide which fairness a machine enforces?

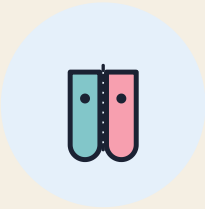
Your Data, Your Rights

23

A video shows someone saying things they never said — flawlessly. How is that possible, and why should it worry all of us?

The generators from the Loom and Kiln (Chapters 15–16) and the scraping from Chapter 21 don't stay in the lab. Point them at real people and the stakes stop being technical and become deeply human.

Before the machinery, three places you have already met the idea —



A **deepfake**: a generated video or voice that convincingly puts words in a real person's mouth — made from images and clips scraped from their public life.

A private dataset leaks, exposing details people never agreed to share. Once out, it can't be pulled back.

Generated misinformation, aimed and repeated, floods feeds — designed to be shared faster than it can be checked.

STEP 1 OF 3

Generation plus your data

Combine powerful generators with scraped pictures and voices, and a convincing fake of a real person becomes cheap to make — that's a deepfake.

STEP 2 OF 3

Privacy and consent at stake

Privacy (control over your information) and **consent** (agreeing to its use) are the human rights this technology strains — often without anyone asking you.

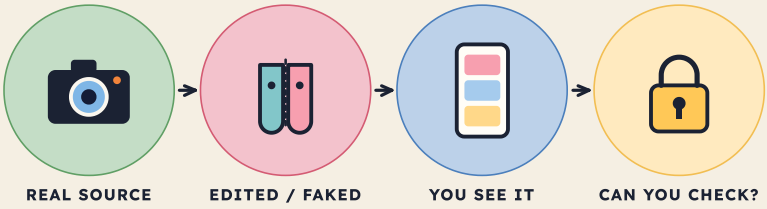
What a viewer can check

The defence is provenance again: a traceable chain — sources, signatures, **watermarks** — that lets a viewer ask “where did this really come from?”

A generated video or voice that fakes a real person has a name:

DEEPFAKE

A deepfake — which is why provenance and consent now matter to everyone.



a chain of custody — detection lags creation

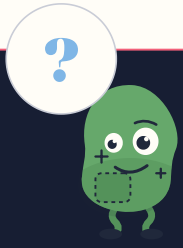
Your mental model. A provenance chain: real source → edited/faked → what you see → can you verify it?

Detection always lags creation. As fakes improve, the tools to spot them race to keep up and often fall behind — so we can't rely on "a detector will catch it". The harder defences are social and legal, not just technical.

WATCH OUT

Myth: "Seeing is believing."

Truth: Not anymore. A realistic video is no longer proof. The new habit is to ask where something came from and whether it can be verified — before believing or sharing it.



Milo looks for where it cracks — because knowing the limits is how you use it well.

TRY THIS

Next time a shocking clip appears, pause before sharing. Ask: who first posted this, when, and can I find it from a second trustworthy source? Practise the pause — it's the whole skill.

You can now explain Deepfake.

THE INTUITION BEHIND IT

In Chapter 16, two machines competed to make fakes ever more convincing. Here those fakes meet real people, with real consequences.

WHERE THIS GOES NEXT

Standards for proving where media came from — and laws to enforce them — are being built right now. Book 3 covers the fight.

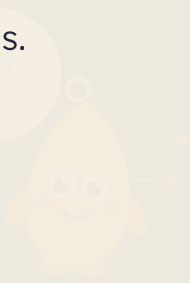
TALK IT OVER

If any video can be faked and any detector can be beaten, what should count as proof that something really happened?

Who Should Decide?

The machine gives its answer. Now — who is responsible for what happens next? This is the question the whole book has been climbing toward.

We've opened every panel of the Works: numbers, learning, layers, generation, reliability, data. One question remains, and no mechanism can answer it. It belongs to us.



YOU ALREADY KNOW THIS

Before the machinery, three places you have already met the idea —

A diagnostic model flags a scan. The *doctor* decides — weighing the model, the patient, and everything the machine can't see.

A risk score advises a judge. The *judge*, not the score, is answerable for the sentence — and must be able to explain it.

A chatbot suggests an answer for your homework. *You* choose whether to use it, check it, or think for yourself.

STEP 1 OF 3

Machines advise; people decide

The pattern that ties the book together is **human-in-the-loop**: the machine proposes, but a person makes the call, holds the values, and owns the outcome.

STEP 2 OF 3

Keep the human accountable

Responsible use means the accountable node is a human who can question, override, and explain — not a black box nobody can answer for.

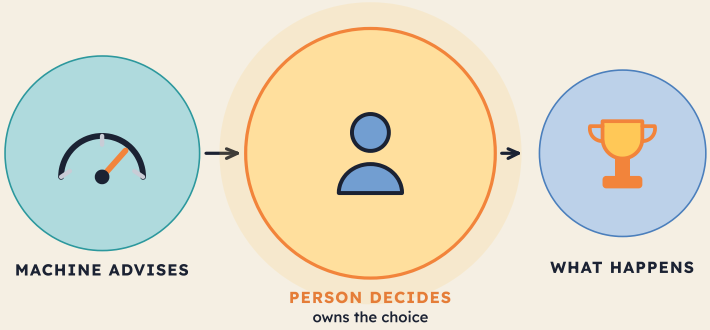
Beware automation bias

The trap is **automation bias**: trusting the machine just because it's a machine. The human in the loop only helps if they stay genuinely free to say no.

Keeping a person responsible for the final call has a name:

HUMAN-IN-THE-LOOP

Human-in-the-loop — machines advise; people decide.



the machine advises; the human is accountable

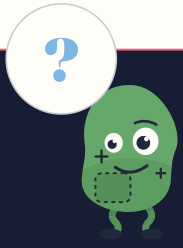
Your mental model. A decision flow with the human as the accountable node — the machine advises, the person decides.

Putting a human “in the loop” isn't a magic fix. Overworked or over-trusting people may rubber-stamp whatever the machine says, and then the human is a fig leaf, not a safeguard. Accountability has to be real, not decorative.

WATCH OUT

Myth: “The algorithm decided, so it's fair and final.”

Truth: An algorithm's output is an input to a human decision. People chose its goal, its data, and whether to follow it — so people remain responsible, every time.



Milo looks for where it cracks — because knowing the limits is how you use it well.

TRY THIS

Pick one decision a machine helps with in your life (a route, a recommendation, an autocorrect). Ask: do I check it, or just obey? Try overriding it once today, on purpose.

You can now explain Human-In-The-Loop.

THE INTUITION BEHIND IT

Every mechanism in this book can advise, but none can decide what matters — that stays a human job. Judgment, fairness, and responsibility are not gaps the machine will someday fill; they are the part that is yours.

WHERE THIS GOES NEXT

Keeping powerful, capable systems aligned with human values — the alignment problem and its governance — is exactly where Book 3 begins.

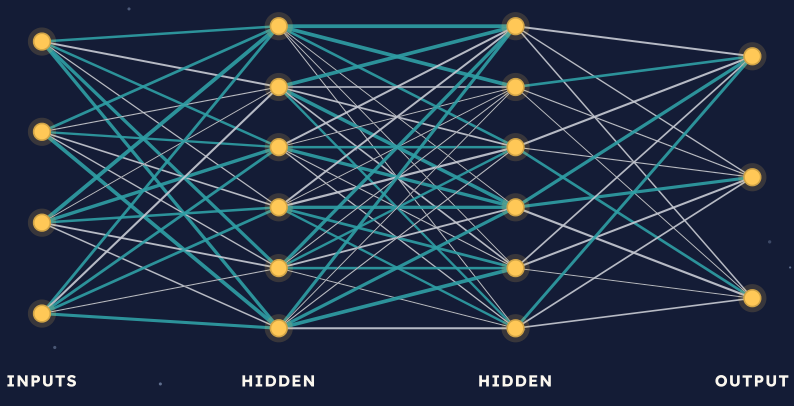
TALK IT OVER

As machines get better at advising, what decisions should *always* stay a human's to make — no matter how good the machine becomes?

The Whole Machine

Every workshop built one part. Here it is, assembled.

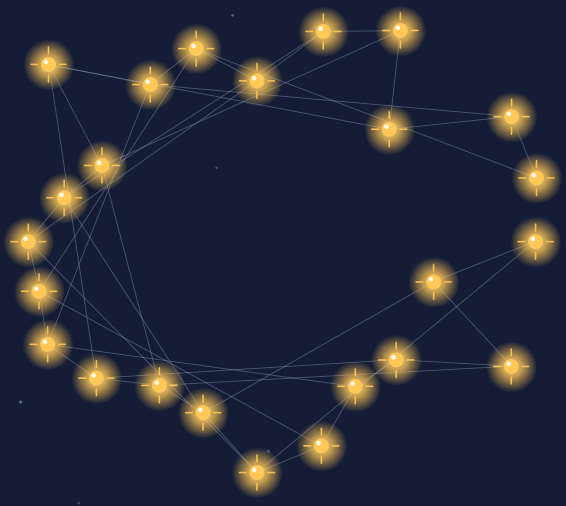
thicker thread = stronger weight



simple parts, stacked deep → complex ideas

every chapter added one part of this one machine

You started by turning the world into numbers, learned to shrink mistakes by rolling downhill, stacked tiny deciders into deep layers that see and focus, wove tokens into language and carved images from noise, tested where it all breaks, and asked where the data — and the responsibility — comes from. It was never magic. It was this machine, one mechanism at a time.



...and the part that stays yours: deciding what it's all for.

The Words You Can Now Use

Each term · what it means · where you met it.

Vector · Chapter 1
A thing turned into an ordered list of numbers, so a machine can store and compare it.

Embedding · Chapter 2
A map of words or things where distance means difference, so meaning becomes a place.

Token · Chapter 3
A sub-word chunk of text — the piece a machine actually reads, between a letter and a word.

Decision Boundary · Chapter 4
The line or curve that separates groups in feature-space; classifying is asking which side a point is on.

Loss · Chapter 5
One number measuring how far a machine's guesses are from the truth; learning means making it smaller.

Gradient Descent · Chapter 6
Lowering the loss by measuring its slope and stepping downhill, over and over.

Overfitting · Chapter 7
When a model memorises its training data instead of learning the general idea, so it fails on new cases.

Self-Supervised · Chapter 8
Learning where the data hides part of itself to make its own answer key — no human labels needed.

Neuron · Chapter 9
The smallest deciding unit: it weights its inputs, sums them, and fires through an activation.

Deep Learning · Chapter 10
Stacking many layers of neurons so each builds on the last, letting simple parts represent complex ideas.

Convolution · Chapter 11
Sliding one small feature-detector across a whole image so the feature is found anywhere it appears.

Attention · Chapter 12
A mechanism that weighs how much each piece of input matters right now and focuses on the important ones.

Language Model · Chapter 13
A machine that writes by repeatedly predicting the next token from the previous ones.

Hallucination · Chapter 14
A confident, fluent, false answer — the next-word machine filling a gap with plausibility instead of truth.

Diffusion · Chapter 15
Making an image by starting from random noise and repeatedly removing a little, guided by a prompt.

Gan · Chapter 16
Two machines — a generator and a discriminator — that improve by competing, producing realism.

Error Types · Chapter 17
The different kinds of mistakes a model makes — false alarms and misses — which matter in different amounts.

Calibration · Chapter 18
A model is calibrated when its stated confidence matches how often it's actually right.

Adversarial Example · Chapter 19
An input changed in a tiny, calculated way that makes a model confidently wrong.

Distribution Shift · Chapter 20
When the real world at use-time differs from the training data, so a model quietly gets worse.

Provenance · Chapter 21
The origin and history of the data a model learned from — who made it and how it was gathered.

Bias · Chapter 22
Unfairness that enters a model through skewed data, labels, features, or goals — and can be amplified.

Deepfake · Chapter 23
A generated video or voice that convincingly fakes a real person, raising privacy and consent at stake.

Human-In-The-Loop · Chapter 24
A setup where a machine advises but a responsible person makes and owns the final decision.

Think Like This

The reasoning moves you used all through the book — now named, so you can use them on purpose.

1. Test an assumption

Ask what the claim is quietly taking for granted, then check just that. (“This is 99% accurate” assumes accuracy is the right measure — is it?)

2. Correlation isn't cause

Two things moving together doesn't mean one caused the other. The model that learned “snow = wolf” confused a coincidence for a reason.

3. Check the base rate

Before trusting a “positive”, ask how common the thing is to begin with. A great test for a rare disease can still be wrong most times it fires.

4. Ask what's measured

A single score hides the shape of the failures. Ask which mistakes it's counting — and which it's ignoring.

5. Consider the counterexample

Don't just collect cases that fit. Hunt for the one that breaks the rule — the husky in the snow, the word on the boundary.

6. Separate fluent from true

Confidence and polish are not evidence. Ask “how could I check this?” especially when it sounds most sure.

For Grown-Ups & Teachers

Part I · The Foundry
Representation: features, vectors, embeddings, tokens, decision boundaries. Everything downstream depends on these — start here.

Part II · The Training Yard
Learning: loss, gradient descent, over/underfitting, the four learning setups. The engine of how models improve.

Part III · The Lattice
Neural networks: the neuron, depth, convolution, attention. From one decider to the architectures behind modern AI.

Part IV · The Loom & the Kiln
Language & generation: next-token prediction, hallucination, diffusion, GANs. How machines write and create — and why they err.

Part V · The Proving Grounds
Reliability: error types, calibration, adversarial examples, distribution shift. Engaging honestly with failure.

Part VI · The Commons
Data & people: provenance, bias, deepfakes, human-in-the-loop. Where the ethics live.

HOW TO USE THIS BOOK

Each chapter ends with a *Talk It Over* question made for discussion — at home or in class. The *Go Deeper* sidebars are optional rungs for keen readers and can be skipped without losing the thread. Every chapter is honest about where the mechanism breaks: that honesty is the point.

WHERE THIS SITS

This book stands on its own — every idea is built from scratch in these pages. It is also the middle of a three-book series: the first introduces these ideas as games for younger readers (ages 6–10), and the third (ages 15–18) carries the mechanisms to the frontier — open problems, debates, and the questions that close the field.

Open the machine and see how it really runs.

Everyone can use AI. This book is for anyone who wants to understand it — the real machinery beneath the magic. How thinking machines turn the world into numbers, learn from their own mistakes, see, focus, write, and create — and exactly where they break. Start anywhere; every idea is built from the ground up.

Turns out writing a book is easier than convincing someone to publish it. So I skipped that part.

I wrote these books for my favourite people. Since you're here, I'm assuming that's you.

They're free until a publisher discovers my existence, offers me a contract, and starts sending royalty cheques.

Enjoy the book. If you like it, drop me a note. If you love it—or if it sparks a new idea for someone you love—Hot Wheels and LEGO remain valid currency in my kingdom.

DIO-STESSO

WHAT IT BUILDS — a real, mechanical understanding of modern AI — representation, learning, neural networks, language and image generation, reliability, and the human choices around it. The second rung of a three-book ladder.

WHAT IT IS NOT — not a coding manual, not hype, not doom. No step-by-step build instructions — and every chapter is honest about what the mechanism can and cannot do.

For intermediate readers of any age — and made for ages 11–15, who'll build with AI. Explanatory, never how-to.